Arrays in DSA with Java

Step-by-Step, Zero to Interview Level

A beginner handbook for internship interviews: array basics, patterns, 20 LeetCode-style problems, 30 MCQs, 25 coding exercises, interview Q&A, edge-case checklist, revision sheet and a 7-day study plan.

All code is Java 17+ and was compiled and executed; every count, trace and answer was machine-checked.
Practice questions are original; they are NOT claimed to be actual Google or Microsoft interview questions.

Contents

Contents 2

1. Array Basics in Java 5

1.1 What is an array? 5
1.2 Declaring and creating arrays 5
1.3 Indexing and length 6
1.4 Reading input and printing output 6
1.5 Traversal: visiting every element 7
1.6 Copying arrays and useful helpers 7

2. Time and Space Complexity for Arrays 9

2.1 What do we measure? 9
2.2 Step-by-step loop analysis 9
2.3 Auxiliary space: where does extra memory come from? 10
2.4 Cost of basic array operations 11

3. Insertion and Deletion 12

3.1 Insertion at an index 12
3.2 Deletion at an index 12

4. The Six Array Patterns 14

4.1 Pattern 1: Two pointers 14
4.2 Pattern 2: Sliding window 15
4.3 Pattern 3: Prefix and suffix sums 16
4.4 Pattern 4: Hashing 17
4.5 Pattern 5: Kadane's algorithm (maximum subarray) 17
4.6 Pattern 6: Binary search 18

5. 2D Arrays (Matrices) 19

5.1 What is a 2D array? 19
5.2 Row-wise and column-wise traversal 19
5.3 Transpose 20
5.4 Rotate a matrix by 90 degrees clockwise 20
5.5 Spiral traversal 21

6. Twenty LeetCode-Style Problems 22

6.1 Problem 1: Maximum Element 22
6.2 Problem 2: Second Largest Distinct Element 24
6.3 Problem 3: Linear Search 25
6.4 Problem 4: Reverse an Array 26
6.5 Problem 5: Check if an Array is Sorted 27

6.6 Problem 6: Remove Duplicates from a Sorted Array . . . 28
6.7 Problem 7: Move Zeroes . . . 29
6.8 Problem 8: Rotate Array . . . 31
6.9 Problem 9: Two Sum . . . 32
6.10 Problem 10: Best Time to Buy and Sell Stock . . . 34
6.11 Problem 11: Maximum Subarray (Kadane's Algorithm) . . . 35
6.12 Problem 12: Merge Sorted Arrays . . . 36
6.13 Problem 13: Majority Element . . . 37
6.14 Problem 14: Missing Number . . . 39
6.15 Problem 15: Contains Duplicate . . . 40
6.16 Problem 16: Product of Array Except Self . . . 41
6.17 Problem 17: Maximum Sum of a Window of Size k . . . 42
6.18 Problem 18: Longest Subarray With a Given Sum . . . 44
6.19 Problem 19: Search in a Rotated Sorted Array . . . 45
6.20 Problem 20: Spiral Matrix . . . 47

7. 30 Multiple-Choice Questions . . . 49

7.1 Beginner level (Questions 1-10) . . . 49
7.2 Intermediate level (Questions 11-20) . . . 50
7.3 Interview level (Questions 21-30) . . . 51

8. Coding Practice: 10 Beginner, 10 Intermediate, 5 Advanced . . . 54

8.1 Beginner problems (B1-B10) . . . 54
8.2 Intermediate problems (I1-I10) . . . 55
8.3 Advanced problems (A1-A5) . . . 56

9. Interview Questions and Answers . . . 58

10. Common Java Mistakes with Arrays . . . 60

10.1 Off-by-one: going past the last index . . . 60
10.2 Thinking b = a makes a copy . . . 60
10.3 Comparing arrays with == . . . 60
10.4 Changing the loop variable of a for-each . . . 60
10.5 Integer overflow in sums . . . 61
10.6 Integer division when computing an average . . . 61
10.7 Arrays.asList with a primitive array . . . 61
10.8 Using a slot of an object array before filling it . . . 61
10.9 Mixing up rows and columns . . . 61
10.10 Sorting an int[] in descending order . . . 62

11. Edge-Case Checklist . . . 63

12. Complexity Tables . . . 64

12.1 The twenty problems . . . 64

12.2 Java collections and arrays . . . 64
12.3 Pattern summary . . . 65

13. One-Page Revision Sheet . . . 66

14. 7-Day Study Plan . . . 67

15. Answer Key . . . 68

15.1 Multiple-choice answers and explanations . . . 69
15.2 Coding practice: solutions . . . 70

1. Array Basics in Java

1.1 What is an array?

An **array** is a row of boxes that all hold the same type of data, placed side by side in memory. Each box has a number called its **index**. In Java the first box has index **0**.

Think of a row of school lockers numbered 0, 1, 2, 3, ... If you know the locker number you can walk straight to it. You do not need to open the other lockers. This is why reading `a[i]` is very fast: it takes the same small time for any `i`.

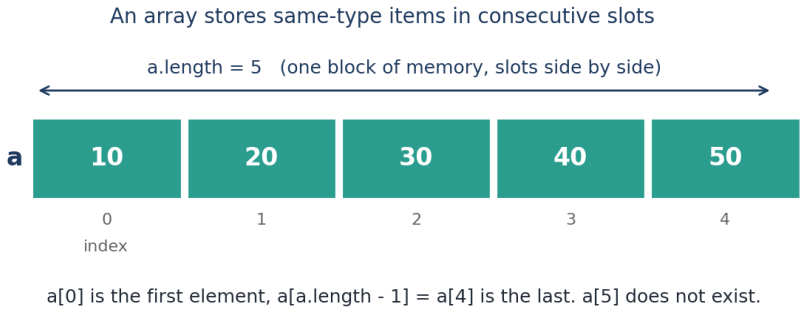


Figure 1.1 An array of five ints. The last valid index is length - 1.

Fact about Java arrays	What it means
Same type	`int[]` holds only ints, `String[]` holds only Strings.
Fixed size	The length is chosen when the array is created and **cannot change**.
Index starts at 0	Valid indexes are 0 to `length - 1`.
Fast access	`a[i]` takes O(1) time.
It is an object	An array variable holds a *reference*. Two variables can point to the same array.

1.2 Declaring and creating arrays

Code	Meaning
`int[] a;`	Declares a reference. No array exists yet.
`int[] a = new int[5];`	Creates 5 slots, all filled with the default value 0.
`int[] a = {2, 3, 5};`	Creates an array of length 3 with these values.
`int[] a = new int[]{2, 3, 5};`	Same as above (useful when passing directly to a method).
`int[][] g = new int[3][4];`	A 2D array: 3 rows and 4 columns.

Default values. When you create an array with `new`, Java fills every slot for you:

Element type	Default value
`int`, `long`, `short`, `byte`	0
`double`, `float`	0.0
`boolean`	false
`char`	the character with code 0 (prints as an invisible character)
`String` and other objects	`null`

```
static void declareDemo() {
    int[] marks = new int[5];                // five ints, every slot starts as 0
```

```
    int[] primes = {2, 3, 5, 7, 11};          // size 5, values given directly
    String[] names = new String[3];           // object slots start as null
    boolean[] flags = new boolean[2];         // false
    double[] prices = new double[2];          // 0.0
    char[] letters = {'a', 'b', 'c'};
    System.out.println(Arrays.toString(marks));
    System.out.println(Arrays.toString(primes));
    System.out.println(Arrays.toString(names));
    System.out.println(Arrays.toString(flags));
    System.out.println(Arrays.toString(prices));
    System.out.println(letters.length);
}
```

Output:

```
[0, 0, 0, 0, 0]
[2, 3, 5, 7, 11]
[null, null, null]
[false, false]
[0.0, 0.0]
3
```

1.3 Indexing and length

`a[i]` reads or writes the element at index `i`. The number of elements is `a.length`. Notice the three different spellings Java uses: `a.length` for arrays (no brackets), `s.length()` for Strings and `list.size()` for lists.

If you use an index outside `0 .. length-1`, Java stops the program with an **ArrayIndexOutOfBoundsException**. This is the most common array error.

```
static void indexDemo() {
    int[] a = {10, 20, 30, 40, 50};
    System.out.println(a[0]);                 // first element
    System.out.println(a[a.length - 1]);      // last element
    a[2] = 99;                                // change one element
    System.out.println(Arrays.toString(a));
    try {
        System.out.println(a[5]);             // valid indexes are 0..4
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Caught: " + e.getMessage());
    }
}
```

```
10
50
[10, 20, 99, 40, 50]
Caught: Index 5 out of bounds for length 5
```

> **Remember**
>
> The **last** element is `a[a.length - 1]`, not `a[a.length]`. A loop that runs `i <= a.length` goes one step too far.

1.4 Reading input and printing output

In coding platforms the first line is usually the length `n`, and the second line holds the `n` numbers. For example the input `5` and `10 20 30 40 50` becomes the array `[10, 20, 30, 40, 50]`.

```
static int[] readArray(Scanner sc) {
    int n = sc.nextInt();                     // first number: the length
    int[] a = new int[n];
    for (int i = 0; i < n; i++) {
        a[i] = sc.nextInt();                  // then n numbers
    }
    return a;
}
```

`Scanner` is easy but slow for very large input (hundreds of thousands of numbers). `BufferedReader` is faster:

```
static int[] readArrayFast(BufferedReader br) throws IOException {
    int n = Integer.parseInt(br.readLine().trim());
    int[] a = new int[n];
    StringTokenizer st = new StringTokenizer(br.readLine());
    for (int i = 0; i < n; i++) {
        a[i] = Integer.parseInt(st.nextToken());
    }
    return a;
}
```

To print, `Arrays.toString(a)` gives the format `[3, 1, 4]`. If you must print many numbers, collect them in a `StringBuilder` and print once.

```
static void printDemo(int[] a) {
    System.out.println(Arrays.toString(a));   // [3, 1, 4]
    StringBuilder sb = new StringBuilder();   // fast way for big outputs
    for (int x : a) sb.append(x).append(' ');
    System.out.println(sb.toString().trim()); // 3 1 4
}
```

```
[10, 20, 30, 40, 50]
[7, 8, 9]
[3, 1, 4]
3 1 4
```

1.5 Traversal: visiting every element

Traversal means visiting the elements one by one. Use an index loop when you need the position `i` (or want to change elements). Use for-each when you only need the values.

```
static void traversalDemo(int[] a) {
    for (int i = 0; i < a.length; i++) {          // index loop: when you need i
        System.out.print(a[i] + " ");
    }
    System.out.println();
    for (int x : a) {                             // for-each: just the values
        System.out.print(x + " ");
    }
    System.out.println();
    for (int i = a.length - 1; i >= 0; i--) {     // backwards
        System.out.print(a[i] + " ");
    }
    System.out.println();
    for (int i = 0; i < a.length; i += 2) {       // every second element
        System.out.print(a[i] + " ");
    }
    System.out.println();
}
```

Output for the array `{1, 2, 3, 4, 5}`:

```
1 2 3 4 5
1 2 3 4 5
5 4 3 2 1
1 3 5
```

Careful: in a for-each loop, changing the loop variable does *not* change the array: `for (int x : a) x = 0;` does nothing to `a`.

1.6 Copying arrays and useful helpers

`int[] b = a;` does **not** copy the array. It makes `b` point to the same array (an *alias*). Changing `b[0]` also changes `a[0]`. To make a real copy use `Arrays.copyOf`. Use `Arrays.equals(x, y)` to compare contents, because `x == y` only checks whether they are the same object.

```
static void utilsDemo() {
    int[] a = {5, 2, 9, 1};
    int[] copy = Arrays.copyOf(a, a.length);          // independent copy
```

```
    int[] part = Arrays.copyOfRange(a, 1, 3);         // indexes 1 and 2
    Arrays.sort(copy);                                // sorts in place
    System.out.println(Arrays.toString(a) + " " + Arrays.toString(copy));
    System.out.println(Arrays.toString(part));
    int[] zeros = new int[4];
    Arrays.fill(zeros, 7);                            // [7, 7, 7, 7]
    System.out.println(Arrays.toString(zeros));
    int[] alias = a;                                  // NOT a copy: same array
    alias[0] = -1;
    System.out.println(a[0] + " " + Arrays.equals(a, alias) + " " + (a == copy));
}
```

```
[5, 2, 9, 1] [1, 2, 5, 9]
[2, 9]
[7, 7, 7, 7]
-1 true false
```

Method	What it does	Time
`Arrays.toString(a)`	text like [1, 2, 3]	O(n)
`Arrays.sort(a)`	sorts ascending, in place	O(n log n)
`Arrays.fill(a, v)`	sets every slot to v	O(n)
`Arrays.copyOf(a, len)`	new copy (pads with zeros if longer)	O(len)
`Arrays.copyOfRange(a, from, to)`	copy of indexes from .. to-1	O(to - from)
`Arrays.equals(a, b)`	true if same length and same values	O(n)
`Arrays.binarySearch(a, v)`	binary search in a sorted array	O(log n)

2. Time and Space Complexity for Arrays

2.1 What do we measure?

Time complexity tells how the number of steps grows when the array length `n` grows. **Space complexity** tells how the extra memory grows. We write them with Big-O: we ignore constants and keep only the biggest term, so 3n + 5 becomes O(n).

For space, we usually give the **auxiliary space**: the *extra* memory your algorithm creates (new arrays, hash maps, big variables), not counting the input array itself. **Total space** = input + auxiliary.

> **Analogy**
> Counting how many pages you flip, not how many seconds it takes. A faster phone does not change the number of pages.

2.2 Step-by-step loop analysis

1. One loop over the array

```
int sum = 0;
for (int i = 0; i < a.length; i++) {
    sum += a[i];
}
```

The variable `i` takes the values 0, 1, ..., n-1, so the body runs **n** times. Each run does a constant amount of work. Time = c x n, which is **O(n)**. Extra memory: `sum` and `i` only, so **O(1)** space.

2. Two loops, one after the other

A loop of n steps followed by another loop of n steps costs n + n = 2n, which is **O(n)**. When loops are *separate*, we **add**. When one loop is *inside* another, we **multiply**.

3. A loop inside a loop (all pairs)

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // work
    }
}
```

For each of the n values of `i`, the inner loop runs n times: n x n = $\mathbf{n^2}$. Example: n = 1,000 gives 1,000,000 steps.

4. The pair loop with j = i + 1 (triangle)

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        // compare a[i] and a[j]
    }
}
```

For `i = 0` the inner loop runs n-1 times, for `i = 1` it runs n-2 times, ..., for `i = n-1` it runs 0 times. Total:

$$(n-1) + (n-2) + \ldots + 1 + 0 = n(n-1)/2$$

Why? Write the sum twice, once forwards and once backwards, and add them column by column. Every column adds up to n-1 and there are n columns, so twice the sum is n(n-1). Half of that is $n(n-1)/2 = n^2/2 - n/2$. Drop the constant 1/2 and the smaller term: **O(n^2)**. For n = 5 there are 4 + 3 + 2 + 1 + 0 = 10 comparisons.

5. Two pointers

```java
int left = 0, right = a.length - 1;
while (left < right) {
    // do something
    left++;     // or right--, or both
}
```

At the start `right - left = n - 1`. Every pass of the loop makes the gap smaller by at least 1, and the loop stops when the gap reaches 0. So there are at most n - 1 passes: **O(n)**. It looks like "two" variables, but each element is visited about once.

6. Sliding window (a while loop inside a for loop)

```java
int left = 0;
for (int right = 0; right < n; right++) {
    // add a[right] to the window
    while (windowIsBad) {
        // remove a[left] from the window
        left++;
    }
}
```

It *looks* like O(n^2) because of the nested loops, but check how often each variable moves. `right` moves exactly n times. `left` only moves forward and can never pass `right`, so it moves at most n times **in total** (not per iteration). The total work is at most n + n = 2n: **O(n)**. This kind of counting is called **amortized analysis**.

7. Halving the search range (binary search)

Each step throws away half of the remaining range: n, n/2, n/4, ..., 1. After k steps the range has about n / 2^k items. It reaches 1 when 2^k is about n, so k is about $\log_2$ n. The loop runs at most **floor($\log_2$ n) + 1** times: **O(log n)**. For n = 1,000,000 that is only 20 steps.

8. A matrix with r rows and c columns

The outer loop runs r times and the inner loop runs c times: **r x c** steps. For a square n x n matrix this is n^2.

2.3 Auxiliary space: where does extra memory come from?

What you create	Extra space	Example
a few `int` variables	O(1)	sum, max, left, right
a new array of length n	O(n)	reversed copy, prefix array, result array
a `HashMap`/`HashSet` of up to n items	O(n)	Two Sum, contains duplicate
a new r x c matrix	O(r x c)	transpose of a non-square matrix
recursion of depth d	O(d)	recursive binary search: d = log n

> **In-place**
> An algorithm is **in place** if it changes the input array itself and uses only O(1) extra space (for example reversing with two pointers). The output array that a problem asks you to *return* is usually not counted, but say so clearly in an interview.

2.4 Cost of basic array operations

Operation	Time	Why
Read / write `a[i]`	O(1)	direct jump to the slot
Search an unsorted array	O(n)	may check every element
Search a sorted array (binary search)	O(log n)	halves the range each step
Find min / max / sum	O(n)	must look at every element
Insert at the end (spare room)	O(1)	no shifting
Insert at index i	O(n)	shift up to n elements right
Delete at index i	O(n)	shift up to n elements left
Delete the last element	O(1)	just reduce the size
Copy the array	O(n)	n writes
Sort	O(n log n)	`Arrays.sort` on `int[]`
Grow the array (new bigger array)	O(n)	copy every element

Java note: `ArrayList` is a dynamic array. `add` at the end is **amortized O(1)** (most adds are cheap, a few adds copy the array, and on average each add is constant). `add(0, x)` and `remove(0)` are O(n).

3. Insertion and Deletion

A Java array has a fixed length, so "inserting" or "deleting" really means **moving elements** inside the array. We keep a variable `size` that says how many slots are in use. The array length is the **capacity**.

3.1 Insertion at an index



Figure 3.1 To insert 25 at index 2, move 40 and 30 one place right (starting from the end), then write 25.

```
// 'size' = how many slots are really used; a.length = capacity
static int insertAt(int[] a, int size, int index, int value) {
    if (size == a.length || index < 0 || index > size) return size;  // no room / bad index
    for (int i = size; i > index; i--) {
        a[i] = a[i - 1];                      // shift right, starting from the end
    }
    a[index] = value;
    return size + 1;                          // new size
}
```

Why start from the end? If you shifted from the front, `a[index + 1] = a[index]` would overwrite `a[index + 1]` before you had copied it. Starting from the end always writes into a slot that is already safe.

Dry run: array `[10, 20, 30, 40, _, _]`, size 4, insert 25 at index 2.

Step	Action	Array after the step
start	size = 4	[10, 20, 30, 40, 0, 0]
i = 4	a[4] = a[3]	[10, 20, 30, 40, 40, 0]
i = 3	a[3] = a[2]	[10, 20, 30, 30, 40, 0]
i = 2	loop stops (i > index is false)	[10, 20, 30, 30, 40, 0]
write	a[2] = 25, size = 5	[10, 20, 25, 30, 40, 0]

Complexity. The loop shifts `size - index` elements.

Where we insert	Shifts	Time
at the end (index = size)	0	O(1)
in the middle	about n/2	O(n)
at the front (index = 0)	n	O(n) (worst case)

Auxiliary space is **O(1)**: only the loop variable. If the array is full you need a new, bigger array and must copy everything: O(n) time and O(n) extra space. `ArrayList` does this for you.

3.2 Deletion at an index

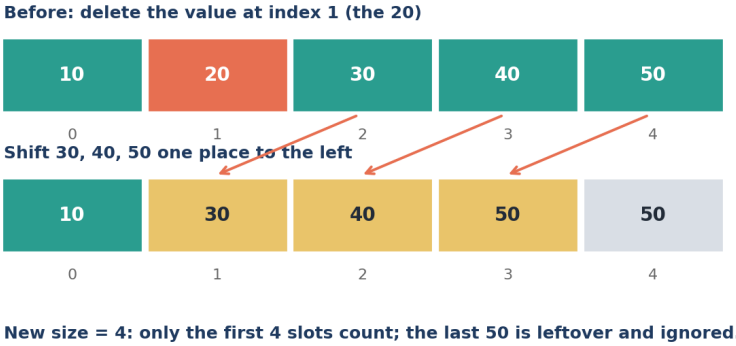


Figure 3.2 To delete index 1, move every later element one place left.

```
static int deleteAt(int[] a, int size, int index) {
    if (index < 0 || index >= size) return size;
    for (int i = index; i < size - 1; i++) {
        a[i] = a[i + 1];                    // shift left, starting at the hole
    }
    return size - 1;
}
```

Dry run: array `[10, 20, 25, 30, 40, 0]`, size 5, delete index 1 (the 20). Here we shift from the *front* of the moving part, because each element moves into a hole that was just freed.

Step	Action	Array after the step
start	size = 5, index = 1	[10, 20, 25, 30, 40, 0]
i = 1	a[1] = a[2]	[10, 25, 25, 30, 40, 0]
i = 2	a[2] = a[3]	[10, 25, 30, 30, 40, 0]
i = 3	a[3] = a[4]	[10, 25, 30, 40, 40, 0]
end	size = 4 (the extra 40 is ignored)	[10, 25, 30, 40] (first 4 slots)

Complexity. Shifts = `size - 1 - index`. Deleting the last element needs no shifting: O(1). Deleting the first element shifts n - 1 elements: O(n). **Delete by value** first needs a linear search (O(n)) and then the shift (O(n)), so it is still O(n). Auxiliary space is O(1).

4. The Six Array Patterns

Most array interview problems are solved with one of a few patterns. Learn to recognise the clue words in the problem, and the right pattern almost chooses itself.

Clue in the problem	Pattern	Typical time / space
sorted array, pair or triplet, palindrome, in-place compaction	Two pointers	O(n) / O(1)
subarray of size k, longest/shortest subarray with a condition (positive numbers)	Sliding window	O(n) / O(1)
many range-sum queries, sum left of / right of i	Prefix and suffix sums	O(n) build, O(1) query / O(n)
have I seen this before? count frequency, find a partner value	Hashing (HashMap / HashSet)	O(n) / O(n)
maximum sum of a contiguous subarray	Kadane's algorithm	O(n) / O(1)
sorted (or rotated sorted) array, find a value fast	Binary search	O(log n) / O(1)

4.1 Pattern 1: Two pointers

Keep two indexes, usually `left` and `right`. Either they start at the two **ends** and move toward each other, or both start at the front and move in the **same direction** (one reads, one writes).

```
// sorted array: find two numbers that add up to target (returns their indexes)
static int[] pairSumSorted(int[] a, int target) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        int sum = a[left] + a[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;             // need a bigger sum
        else right--;                         // need a smaller sum
    }
    return new int[]{-1, -1};
}
```

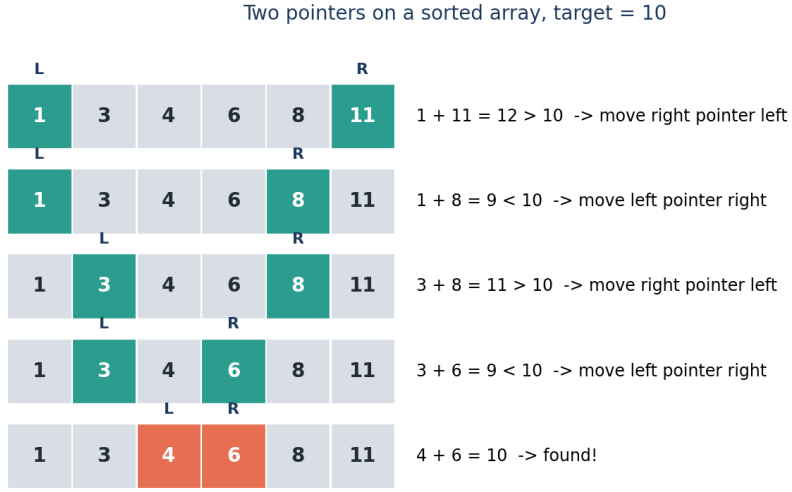


Figure 4.1 Pair sum on the sorted array [1, 3, 4, 6, 8, 11] with target 10. The answer is indexes 2 and 3.

Why moving a pointer is safe. If `a[left] + a[right] < target`, then `a[left]` paired with *any* element at or before `right` is even smaller (the array is sorted), so `a[left]` can never be part of the answer and we can discard it. The same logic discards `a[right]` when the sum is too big. Each step throws one element away, so after at most n - 1 steps we are done.

Complexity

Time O(n) (the gap right - left shrinks by 1 every step). **Auxiliary space O(1).**

Brute force checks all pairs: O(n^2).

Typical uses: pair sum in a sorted array, palindrome check, reverse, container with most water, remove duplicates, move zeroes, merge two sorted arrays, 3Sum.

4.2 Pattern 2: Sliding window

A **window** is a block of consecutive elements `a[left..right]`. Instead of recomputing the window's sum from scratch each time, we **slide** it: add the new element on the right, remove the old element on the left.

Fixed-size window (size k)

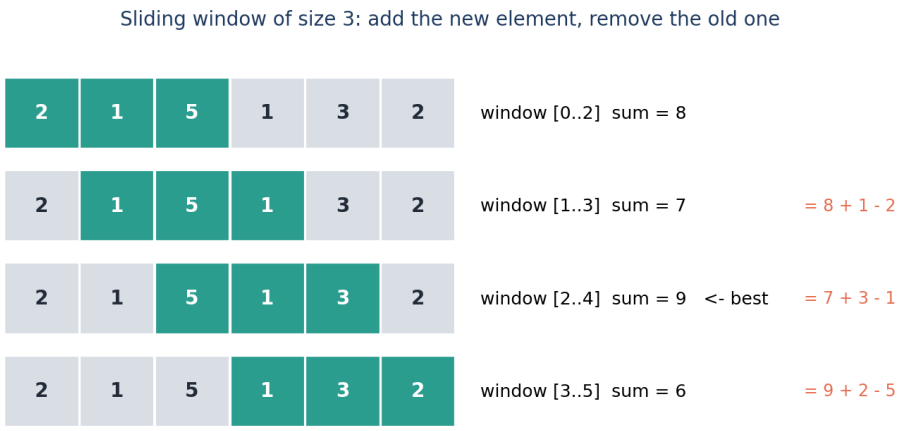


Figure 4.2 Fixed window of size 3. Each slide costs only one addition and one subtraction.

Problem 17 solves this in full. The brute force recomputes k numbers for each of n - k + 1 windows: O(n x k). Sliding makes it **O(n)**.

Variable-size window (all numbers positive)

Grow the window on the right. When the rule is broken, shrink it from the left until it is valid again.

```
// longest subarray whose sum is <= limit (all numbers are positive)
static int longestWindowAtMost(int[] a, int limit) {
    int left = 0, sum = 0, best = 0;
    for (int right = 0; right < a.length; right++) {
        sum += a[right];                    // the window grows on the right
        while (sum > limit) {
            sum -= a[left];                 // shrink from the left
            left++;
        }
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Dry run: array `[2, 1, 3, 1, 1, 4]`, limit 6. (The sum column shows the sum *after* any shrinking.)

right	a[right]	sum after adding	shrink?	window	length	best
0	2	2	no	[0..0]	1	1
1	1	3	no	[0..1]	2	2
2	3	6	no	[0..2]	3	3
3	1	5	drop a[0]=2	[1..3]	3	3
4	1	6	no	[1..4]	4	4

right	a[right]	sum after adding	shrink?	window	length	best
5	4	6	drop a[1]=1, drop a[2]=3	[3..5]	3	4

> **Complexity**
>
> **Time O(n)**: `right` moves n times and `left` moves at most n times in total, so at most 2n steps.
> **Auxiliary space O(1).**
>
> This version needs **positive** numbers. With negative numbers, shrinking does not always reduce the sum; use prefix sums and hashing instead (Problem 18).

4.3 Pattern 3: Prefix and suffix sums

A **prefix sum array** stores running totals: `prefix[i]` is the sum of the first `i` elements. We make the array one slot longer than `a` so that `prefix[0] = 0` means "the sum of nothing". Then any range sum is one subtraction:

sum of a[l..r] = prefix[r + 1] - prefix[l]

```
static int[] buildPrefix(int[] a) {
    int[] prefix = new int[a.length + 1];     // prefix[i] = a[0] + ... + a[i-1]
    for (int i = 0; i < a.length; i++) {
        prefix[i + 1] = prefix[i] + a[i];
    }
    return prefix;
}

static int rangeSum(int[] prefix, int l, int r) {      // sum of a[l..r], inclusive
    return prefix[r + 1] - prefix[l];
}
```

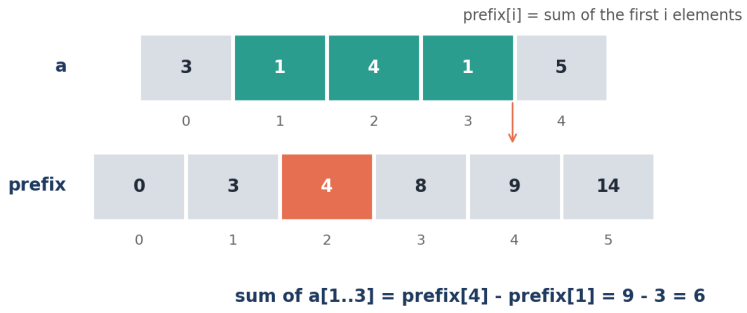


Figure 4.3 The sum of a[1..3] is prefix[4] - prefix[1] = 9 - 3 = 6.

Dry run for `a = [3, 1, 4, 1, 5]`:

i	a[i]	prefix[i+1] = prefix[i] + a[i]
0	3	0 + 3 = 3
1	1	3 + 1 = 4
2	4	4 + 4 = 8
3	1	8 + 1 = 9
4	5	9 + 5 = 14

```
[0, 3, 4, 8, 9, 14] 6
[14, 11, 10, 6, 5, 0]
```

The first line is the prefix array and the range sum of `a[1..3]`. The second line is the **suffix sum** array: `suffix[i]` is the sum from `a[i]` to the end.

```java
static int[] buildSuffix(int[] a) {
    int[] suffix = new int[a.length + 1];     // suffix[i] = a[i] + ... + a[n-1]
    for (int i = a.length - 1; i >= 0; i--) {
        suffix[i] = suffix[i + 1] + a[i];
    }
    return suffix;
}
```

Complexity

Build: O(n) time, O(n) space. Each query: O(1). Without prefix sums, q queries cost O(n x q).

Used in: range sum queries, equilibrium index, product except self (prefix and suffix *products*), subarray sum equals k.

4.4 Pattern 4: Hashing

A `HashMap` or `HashSet` can answer "have I seen this value?" in **O(1) on average**. We trade memory for speed: O(n) extra space to turn an $O(n^2)$ search into an O(n) one.

```java
static void frequencyDemo(int[] a) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : a) {
        freq.put(x, freq.getOrDefault(x, 0) + 1);       // count each value
    }
    System.out.println(new TreeMap<>(freq));            // TreeMap only to print in order
    int[] count = new int[10];                          // faster when values are 0..9
    for (int x : a) count[x]++;
    System.out.println(Arrays.toString(count));
}
```

Output for `{3, 1, 3, 3, 7, 1}`:

```
{1=2, 3=3, 7=1}
[0, 2, 0, 3, 0, 0, 0, 1, 0, 0]
```

- `HashMap<Integer,Integer>` maps value to count (or value to index).
- `HashSet<Integer>` only remembers which values exist; `add` returns `false` if the value was already there.
- If the values are small non-negative numbers (for example 0..9), a plain `int[] count` array is simpler and faster than a map.
- A HashMap does **not** keep order. Do not expect it to print in insertion order.

Complexity

Time O(1) per operation on average, so O(n) for n operations. Space O(n). (In the rare worst case, many keys collide; Java 8+ keeps this at O(log n) per operation.)

4.5 Pattern 5: Kadane's algorithm (maximum subarray)

A **subarray** is a continuous part of an array. Kadane's algorithm finds the biggest possible sum of a subarray in one pass. At each position `i` we ask one question: *is it better to extend the best subarray that ends at i-1, or to start a new subarray at i?*

current = max(a[i], current + a[i]) best = max(best, current)

If the running sum `current` has become negative, carrying it forward only hurts, so we throw it away and restart at `a[i]`. The full code, a dry run and the proof idea are in Problem 11.

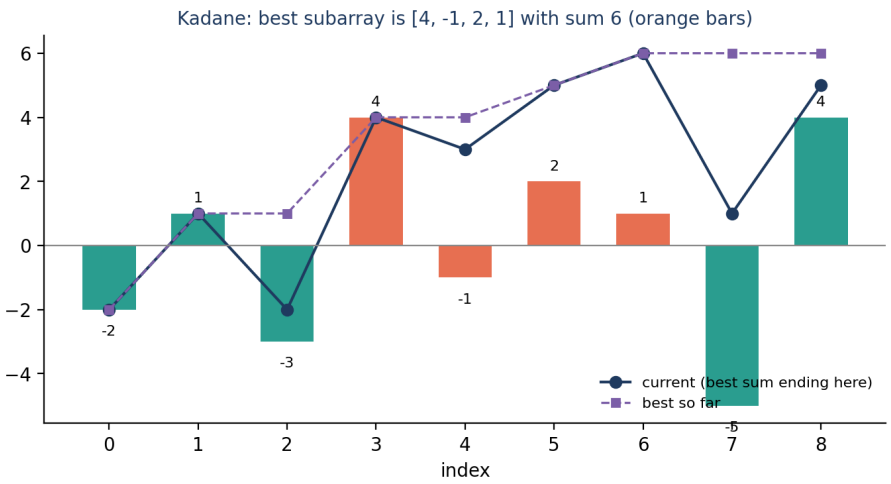


Figure 4.4 The line shows `current`. It restarts whenever carrying the old sum is worse than starting fresh.

4.6 Pattern 6: Binary search

Binary search works on a **sorted** array. Look at the middle element: if it is the target you are done; if it is too small, the target can only be on the right, so discard the left half; otherwise discard the right half.

```
static int binarySearch(int[] a, int target) {          // a must be sorted
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;                   // safe from overflow
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Dry run: array `[2, 5, 8, 12, 16, 23]`, target 12.

Step	lo	hi	mid = lo + (hi-lo)/2	a[mid]	Decision
1	0	5	2	8	8 < 12: lo = mid + 1 = 3
2	3	5	4	16	16 > 12: hi = mid - 1 = 3
3	3	3	3	12	found: return 3

- Why `lo + (hi - lo) / 2` and not `(lo + hi) / 2`? When `lo + hi` is bigger than 2,147,483,647 the sum overflows to a negative number. The first form never overflows.
- Use `while (lo <= hi)` with `lo = mid + 1` and `hi = mid - 1`. Mixing styles (for example `hi = mid`) can make the loop run forever.
- The array must be sorted. Binary search on unsorted data gives wrong answers, not errors.

Complexity

Time O(log n): at most floor($\log_2$ n) + 1 steps (20 steps for 1,000,000 items). **Auxiliary space O(1)** for the loop version.

5. 2D Arrays (Matrices)

5.1 What is a 2D array?

A 2D array is an array of arrays. We picture it as a table with **rows** and **columns**. `m[r][c]` means *row r, column c*. Both indexes start at 0.

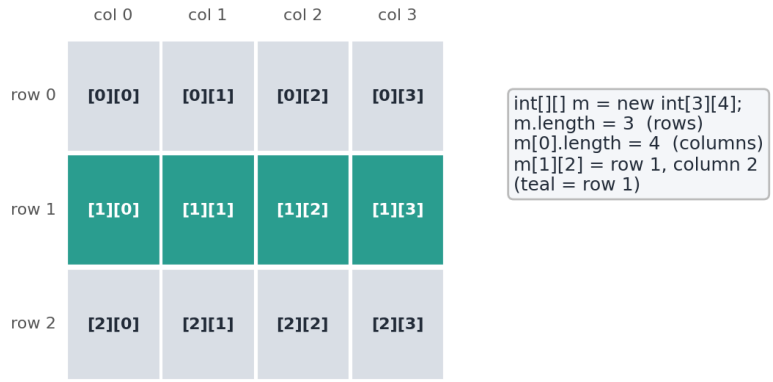


Figure 5.1 A 3 x 4 matrix. `m.length` is the number of rows; `m[0].length` is the number of columns.

```
static void matrixDemo() {
    int[][] m = new int[3][4];                // 3 rows, 4 columns, all 0
    int[][] g = {{1, 2, 3}, {4, 5, 6}};       // 2 rows, 3 columns
    System.out.println(g.length + " rows, " + g[0].length + " columns");
    System.out.println(g[1][2]);              // row 1, column 2 -> 6
    for (int r = 0; r < m.length; r++) {
        for (int c = 0; c < m[r].length; c++) {
            m[r][c] = r * 4 + c;              // row-major traversal: row by row
        }
    }
    System.out.println(Arrays.deepToString(m));
}
```

```
2 rows, 3 columns
6
[[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]]
```

> **Remember**
>
> Rows first, columns second: `m[row][column]`. A common bug is swapping them, which fails on non-square matrices with an ArrayIndexOutOfBoundsException.

5.2 Row-wise and column-wise traversal

Looping `r` on the outside and `c` on the inside visits the matrix **row by row** (row-major order). Swapping the loops visits it **column by column**. Both take **O(r x c)** time and O(1) extra space (not counting the answer arrays).

```
static int[] rowSums(int[][] m) {
    int[] res = new int[m.length];
    for (int r = 0; r < m.length; r++) {
        for (int c = 0; c < m[r].length; c++) {
            res[r] += m[r][c];                // row by row (row-major order)
        }
    }
    return res;
}

static int[] colSums(int[][] m) {
    int[] res = new int[m[0].length];
    for (int c = 0; c < m[0].length; c++) {
```

```
        for (int r = 0; r < m.length; r++) {
            res[c] += m[r][c];                // column by column (column-major order)
        }
    }
    return res;
}
```

For `{{1, 2, 3}, {4, 5, 6}}` the row sums are `[6, 15]` and the column sums are `[5, 7, 9]`:

```
[6, 15] [5, 7, 9]
```

5.3 Transpose

The **transpose** flips the matrix over its main diagonal: the element at row `r`, column `c` moves to row `c`, column `r`. An r x c matrix becomes a c x r matrix.

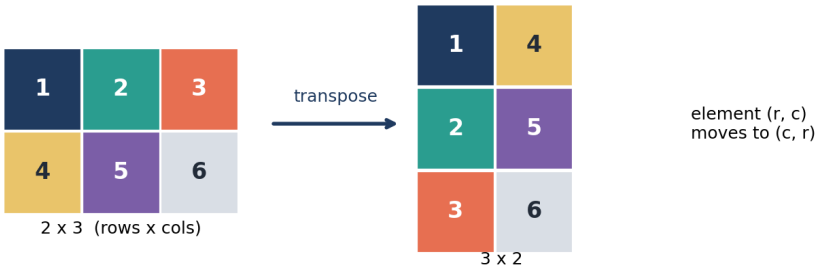


Figure 5.2 Transpose of a 2 x 3 matrix is a 3 x 2 matrix.

```
static int[][] transpose(int[][] m) {         // works for any rows x cols
    int rows = m.length, cols = m[0].length;
    int[][] t = new int[cols][rows];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            t[c][r] = m[r][c];                // (r, c) moves to (c, r)
        }
    }
    return t;
}

static void transposeInPlace(int[][] m) {     // square matrices only
    int n = m.length;
    for (int r = 0; r < n; r++) {
        for (int c = r + 1; c < n; c++) {     // only above the diagonal
            int tmp = m[r][c]; m[r][c] = m[c][r]; m[c][r] = tmp;
        }
    }
}
```

```
[[1, 4], [2, 5], [3, 6]]
[[1, 4, 7], [2, 5, 8], [3, 6, 9]]
```

Complexity. `transpose` visits every cell once: **O(r x c) time and O(r x c) space** for the new matrix. `transposeInPlace` works only for square matrices. It swaps each pair (r, c) and (c, r) once, using only cells above the diagonal. The number of swaps is (n-1) + (n-2) + ... + 1 = n(n-1)/2, so it is **O(n^2) time and O(1) space**.

5.4 Rotate a matrix by 90 degrees clockwise

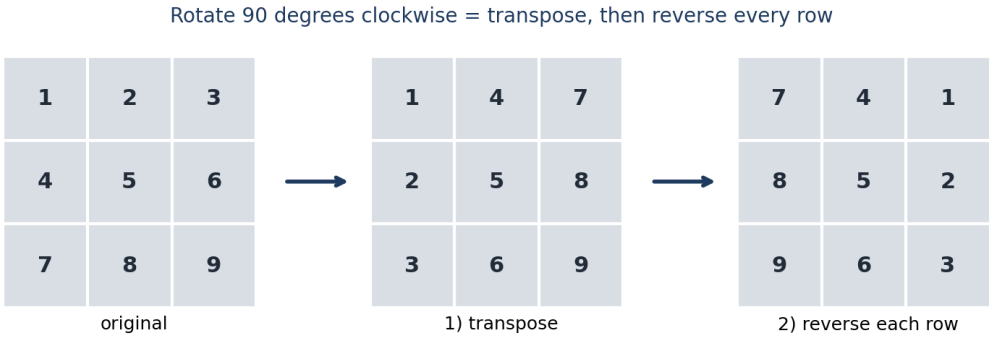


Figure 5.3 Rotate clockwise = transpose, then reverse each row.

```java
static void rotateClockwise(int[][] m) {      // n x n matrix, in place
    int n = m.length;
    transposeInPlace(m);                      // step 1: swap across the diagonal
    for (int r = 0; r < n; r++) {             // step 2: reverse every row
        int l = 0, h = n - 1;
        while (l < h) {
            int tmp = m[r][l]; m[r][l] = m[r][h]; m[r][h] = tmp;
            l++; h--;
        }
    }
}
```

```
[[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

Why it works. After a clockwise rotation, the element at (r, c) lands at (c, n-1-r). The transpose moves it to (c, r). Reversing a row maps column index r to n-1-r, giving (c, n-1-r). That is exactly where it should go.

For a **counter-clockwise** rotation, reverse each row first and then transpose (or transpose, then reverse each column).

> **Complexity**
> **Time O(n^2)** (n(n-1)/2 swaps for the transpose plus about n^2/2 swaps for the reversals). **Auxiliary space O(1)**: no second matrix is needed.

5.5 Spiral traversal

Walking around the matrix in a spiral is Problem 20, with a full explanation and a dry run.

6. Twenty LeetCode-Style Problems

Each problem has the same layout: statement, examples, constraints, brute force (when it makes sense), optimized Java code, a dry run, complexity, edge cases and an explanation. All code was compiled and run, and every optimized solution was tested against its brute-force version on thousands of random arrays.

These are **original practice problems** written in the style of common interview topics. They are not claimed to be actual questions from Google, Microsoft or any other company.

#	Problem	Main idea	Best time	Aux space
1	Maximum	one pass	O(n)	O(1)
2	Second largest	track two values	O(n)	O(1)
3	Linear search	scan until found	O(n)	O(1)
4	Reverse	two pointers	O(n)	O(1)
5	Sorted check	compare neighbours	O(n)	O(1)
6	Remove duplicates	read/write pointers	O(n)	O(1)
7	Move zeroes	read/write pointers	O(n)	O(1)
8	Rotate array	three reversals	O(n)	O(1)
9	Two Sum	hash map	O(n)	O(n)
10	Stock profit	min so far	O(n)	O(1)
11	Kadane	extend or restart	O(n)	O(1)
12	Merge sorted arrays	fill from the back	O(m+n)	O(1)
13	Majority element	Boyer-Moore voting	O(n)	O(1)
14	Missing number	sum formula	O(n)	O(1)
15	Contains duplicate	hash set	O(n)	O(n)
16	Product except self	prefix x suffix	O(n)	O(1)*
17	Max sum window	sliding window	O(n)	O(1)
18	Longest subarray, sum = target	prefix sums + hash	O(n)	O(n)
19	Rotated array search	binary search	O(log n)	O(1)
20	Spiral matrix	shrinking borders	O(r x c)	O(1)*

(*) not counting the output that the problem asks us to return.

6.1 Problem 1: Maximum Element

Problem statement

Given an integer array `nums`, return the largest value in it.

Examples

Input	Output	Why
`nums = [3, 9, 2, 7]`	`9`	9 is the biggest
`nums = [-5, -2, -9]`	`-2`	all negative: -2 is the biggest
`nums = [4]`	`4`	one element

Constraints

- $1 <= n <= 10^5$
- $-10^9 <= nums[i] <= 10^9$

Brute-force approach

For every element, check that nobody else is bigger. That compares each element with all others: n x n steps.

```
static int maxBrute(int[] a) {
    for (int i = 0; i < a.length; i++) {
        boolean isMax = true;
        for (int j = 0; j < a.length; j++) {
            if (a[j] > a[i]) { isMax = false; break; }   // someone is bigger
        }
        if (isMax) return a[i];
    }
    throw new IllegalArgumentException("empty array");
}
```

Optimized Java solution

Remember the biggest value seen so far. Look at each element once; if it is bigger, replace the remembered value. Start with `a[0]`, **not 0**, so that all-negative arrays work. (For the minimum, flip `>` to `<`.)

```
static int max(int[] a) {
    int best = a[0];                          // start with the first element
    for (int i = 1; i < a.length; i++) {
        if (a[i] > best) best = a[i];
    }
    return best;
}
```

Dry run

i	a[i]	best before	a[i] > best?	best after
start	3	-	-	3
1	9	3	yes	9
2	2	9	no	9
3	7	9	no	9

Complexity

Approach	Time	Auxiliary space
Brute force	$O(n^2)$	O(1)
One pass	O(n)	O(1)

Edge cases

- Single element: the loop body never runs and `a[0]` is returned.
- All numbers negative: starting from 0 would wrongly return 0.
- Maximum appears more than once: fine, the value is the same.
- Empty array: not allowed by the constraints (`a[0]` would throw).

Explanation

The one-pass method keeps one variable, `best`, which always holds the maximum of the part of the array already seen. Each element is compared exactly once, so the time is n - 1 comparisons, which is O(n). We cannot do better than O(n) because any element might be the answer, so every element must be looked at.

6.2 Problem 2: Second Largest Distinct Element

Problem statement

Return the second largest **distinct** value in `nums`. If there is no such value (for example all elements are equal), return `null`.

Examples

Input	Output	Why
`[12, 35, 1, 10, 34, 1]`	`34`	largest 35, next different value 34
`[7, 7, 7]`	`null`	only one distinct value
`[5, 5, 3]`	`3`	the second 5 is not distinct

Constraints

- 1 <= n <= 10^5
- any int values, including negatives and `Integer.MIN_VALUE`

Brute-force approach

Sort a copy, then walk from the end and return the first value that is different from the maximum. Sorting costs O(n log n).

```
static Integer secondLargestBrute(int[] a) {
    int[] s = a.clone();
    Arrays.sort(s);                          // O(n log n)
    for (int i = s.length - 2; i >= 0; i--) {
        if (s[i] != s[s.length - 1]) return s[i];   // first value different from the max
    }
    return null;
}
```

Optimized Java solution

Keep two variables, `first` (largest) and `second` (second largest distinct). When a bigger value arrives, the old `first` becomes `second`. An equal value is ignored. We use `long` variables starting at `Long.MIN_VALUE`, which is smaller than every `int`, so even `Integer.MIN_VALUE` works as a real answer.

```
static Integer secondLargest(int[] a) {
    long first = Long.MIN_VALUE, second = Long.MIN_VALUE;   // smaller than any int
    for (int x : a) {
        if (x > first) {
            second = first;                  // old maximum becomes second
            first = x;
        } else if (x < first && x > second) {
            second = x;                      // new second (equal values are skipped)
        }
    }
    return second == Long.MIN_VALUE ? null : (int) second;
}
```

Dry run

x	first	second	action
12	12	-inf	new maximum: second = old first, first = x
35	35	12	new maximum: second = old first, first = x
1	35	12	ignore
10	35	12	ignore
34	35	34	new second largest
1	35	34	ignore

Complexity

Approach	Time	Auxiliary space
Sort and scan	O(n log n)	O(n) (copy)
One pass, two variables	O(n)	O(1)

Edge cases

- `[7, 7, 7]` and `[9]`: return null.
- `[5, 5, 3]`: the duplicate maximum must not become the second answer, hence `x < first`.
- Array containing `Integer.MIN_VALUE`: a plain `int` sentinel would confuse it with "not found".
- Negative numbers.

Explanation

At every step the invariant is: `first` is the maximum seen and `second` is the largest value seen that is strictly smaller than `first`. Case 1 (`x > first`) pushes the old maximum down. Case 2 (`first > x > second`) improves `second`. All other values cannot change either variable, so they are ignored.

6.3 Problem 3: Linear Search

Problem statement

Given an array `nums` and a value `target`, return the **first** index where `target` appears, or `-1` if it does not appear.

Examples

Input	Output	Why
`[4, 8, 15, 16, 23]`, target 16	`3`	a[3] = 16
`[4, 8]`, target 5	`-1`	5 is not in the array

Constraints

- 0 <= n <= 10^5
- array may be unsorted

Brute-force approach

Linear search *is* the brute-force method: there is no simpler approach for an unsorted array. If the array is **sorted**, use binary search (Section 4.7) for O(log n).

Optimized Java solution

Check elements from left to right and return as soon as one matches.

```
static int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;         // first match
    }
    return -1;                                // not found
}
```

Dry run

i	a[i]	a[i] == 16 ?
0	4	no
1	8	no
2	15	no
3	16	yes -> return 3

Complexity

Approach	Time	Auxiliary space
Linear search (best case: first element)	O(1)	O(1)
Linear search (worst case: last or missing)	O(n)	O(1)
Binary search (sorted array only)	O(log n)	O(1)

Edge cases

- Empty array returns -1.
- Duplicates: the first index is returned because we stop at the first match.
- Target at the last index, or missing: n comparisons.

Explanation

The loop checks `a[0]`, `a[1]`, ... until it finds the target. The number of comparisons depends on where the target is: 1 if it is first, n if it is last or missing. On average, if the target is equally likely at any position, we do (1 + 2 + ... + n) / n = (n + 1) / 2 comparisons, still O(n).

6.4 Problem 4: Reverse an Array

Problem statement

Reverse the array `nums` **in place** (change the same array, do not return a new one).

Examples

Input	Output	Why
`[1, 2, 3, 4, 5]`	`[5, 4, 3, 2, 1]`	first and last swap, then second and second-last
`[1, 2]`	`[2, 1]`	one swap

Constraints

- $0 <= n <= 10^5$

Brute-force approach

Create a new array and copy the elements in reverse order. It is simple but uses O(n) extra memory.

```java
static int[] reverseCopy(int[] a) {
    int[] r = new int[a.length];              // extra array: O(n) space
    for (int i = 0; i < a.length; i++) {
        r[i] = a[a.length - 1 - i];
    }
    return r;
}
```

Optimized Java solution

Use two pointers at the two ends. Swap the elements and move the pointers toward each other until they meet.

```java
static void reverse(int[] a) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        int tmp = a[left];
        a[left] = a[right];
        a[right] = tmp;
        left++;
        right--;
    }
}
```

Dry run

left	right	action	array
0	4	swap a[0] and a[4]	[5, 2, 3, 4, 1]
1	3	swap a[1] and a[3]	[5, 4, 3, 2, 1]
2	2	left < right is false: stop	[5, 4, 3, 2, 1]

Complexity

Approach	Time	Auxiliary space
New array	O(n)	O(n)
Two pointers, in place	O(n)	O(1)

Edge cases

- Empty array or one element: the loop does not run.
- Odd length: the middle element stays where it is (pointers meet).
- Even length: all elements are swapped in pairs.

Explanation

Each swap fixes two elements for good, so about n/2 swaps are needed. That is O(n) time. We only store `left`, `right` and one temporary variable, so the extra space is O(1).

6.5 Problem 5: Check if an Array is Sorted

Problem statement

Return `true` if `nums` is sorted in **non-decreasing** order (each element is greater than or equal to the previous one).

Examples

Input	Output	Why
`[1, 2, 2, 5]`	`true`	equal neighbours are allowed
`[1, 3, 2]`	`false`	3 > 2
`[]`	`true`	nothing is out of order

Constraints

- 0 <= n <= 10^5

Brute-force approach

Compare every pair `(i, j)` with `i < j`. Checking all pairs costs n(n-1)/2 comparisons.

```
static boolean isSortedBrute(int[] a) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] > a[j]) return false;    // some pair is out of order
    return true;
}
```

Optimized Java solution

If every neighbour pair is in order, the whole array is in order (order is transitive). So only compare `a[i-1]` and `a[i]` and stop at the first violation.

```
static boolean isSorted(int[] a) {
    for (int i = 1; i < a.length; i++) {
        if (a[i - 1] > a[i]) return false;    // one bad neighbour is enough
    }
```

```
    return true;
}
```

Dry run

i	a[i-1]	a[i]	a[i-1] <= a[i] ?
1	2	4	yes
2	4	4	yes
3	4	7	yes
4	7	5	NO -> return false

Complexity

Approach	Time	Auxiliary space
All pairs	O(n^2)	O(1)
Neighbours only	O(n)	O(1)

Edge cases

- Empty and single-element arrays are sorted.
- Duplicates: use `>` for the failure test, not `>=`.
- Descending arrays fail at the first pair.

Explanation

If `a[0] <= a[1]`, `a[1] <= a[2]`, ... , `a[n-2] <= a[n-1]`, then chaining the inequalities gives `a[i] <= a[j]` for every `i < j`. So checking n - 1 neighbour pairs is enough.

6.6 Problem 6: Remove Duplicates from a Sorted Array

Problem statement

`nums` is sorted in non-decreasing order. Remove duplicates **in place** so that each value appears once, keeping the order. Return `k`, the number of unique values. The first `k` slots of `nums` must hold the answer; the rest does not matter.

Examples

Input	Output	Why
`[1, 1, 2, 2, 2, 3]`	`k = 3`, `[1, 2, 3, ...]`	three distinct values
`[5]`	`k = 1`	already unique

Constraints

- 0 <= n <= 10^5
- `nums` is sorted

Brute-force approach

Copy each element into a helper array only if it is not already there (a nested scan checks this). That is O(n^2) and ignores the fact that the array is sorted.

```
static int removeDuplicatesBrute(int[] a) {
    int[] unique = new int[a.length];
    int k = 0;
    for (int x : a) {
        boolean seen = false;
        for (int j = 0; j < k; j++) {
            if (unique[j] == x) { seen = true; break; }
        }
        if (!seen) unique[k++] = x;
```

```
    }
    System.arraycopy(unique, 0, a, 0, k);
    return k;
}
```

Optimized Java solution

Because the array is sorted, duplicates sit next to each other. Use a **write pointer** `k`: `a[0..k-1]` is the unique answer built so far. For each `a[i]`, if it differs from the last kept value `a[k-1]`, copy it to `a[k]`.

```
static int removeDuplicates(int[] a) {
    if (a.length == 0) return 0;
    int k = 1;                                  // a[0..k-1] is the answer so far
    for (int i = 1; i < a.length; i++) {
        if (a[i] != a[k - 1]) {                 // a new value
            a[k] = a[i];
            k++;
        }
    }
    return k;
}
```

Dry run

i	a[i]	action	k	answer so far
start	-	-	1	[1]
1	1	duplicate: skip	1	[1]
2	2	new value: copy to a[k], k++	2	[1, 2]
3	2	duplicate: skip	2	[1, 2]
4	2	duplicate: skip	2	[1, 2]
5	3	new value: copy to a[k], k++	3	[1, 2, 3]

Complexity

Approach	Time	Auxiliary space
Helper array + nested scan	$O(n^2)$	O(n)
Read/write pointers	O(n)	O(1)

Edge cases

- Empty array: return 0.
- All elements equal: k = 1.
- No duplicates: nothing changes, k = n.
- Negative numbers work the same way.

Explanation

The read pointer `i` visits each element once. The write pointer `k` only moves when a new value appears, so `k <= i` always, and we never overwrite an element we have not read yet. One pass: O(n) time and O(1) space.

6.7 Problem 7: Move Zeroes

Problem statement

Move all `0`s to the end of `nums` while keeping the relative order of the non-zero elements. Do it **in place**.

Examples

Input	Output	Why
`[0, 1, 0, 3, 12]`	`[1, 3, 12, 0, 0]`	non-zeros keep order 1, 3, 12
`[0, 0]`	`[0, 0]`	nothing to move

Constraints

- $1 <= n <= 10^5$

Brute-force approach

Copy the non-zero values into a new array (zeros are the default), then copy back. O(n) time but O(n) extra space.

```
static void moveZeroesBrute(int[] a) {
    int[] tmp = new int[a.length];            // zeros by default
    int k = 0;
    for (int x : a) {
        if (x != 0) tmp[k++] = x;
    }
    System.arraycopy(tmp, 0, a, 0, a.length);
}
```

Optimized Java solution

Use a write pointer for the next position of a non-zero value. Scan with a read pointer; whenever `a[read]` is non-zero, swap it with `a[write]` and advance `write`. All zeros are pushed behind the write pointer automatically.

```
static void moveZeroes(int[] a) {
    int write = 0;                            // next slot for a non-zero value
    for (int read = 0; read < a.length; read++) {
        if (a[read] != 0) {
            int tmp = a[write];
            a[write] = a[read];
            a[read] = tmp;
            write++;
        }
    }
}
```

Dry run

read	action	write	array
0	zero: skip	0	[0, 1, 0, 3, 12]
1	swap a[0] and a[1]	1	[1, 0, 0, 3, 12]
2	zero: skip	1	[1, 0, 0, 3, 12]
3	swap a[1] and a[3]	2	[1, 3, 0, 0, 12]
4	swap a[2] and a[4]	3	[1, 3, 12, 0, 0]

Complexity

Approach	Time	Auxiliary space
Extra array	O(n)	O(n)
Swap with write pointer	O(n)	O(1)

Edge cases

- All zeros: nothing swaps.
- No zeros: every element swaps with itself (harmless).
- Zero already at the end.

- Order of non-zero numbers must not change (do not sort).

Explanation

Invariant: after each step, `a[0..write-1]` holds the non-zero values seen so far in their original order, and `a[write..read]` holds only zeros. Swapping a non-zero from `read` with the first zero at `write` keeps this true. One pass, no extra memory.

6.8 Problem 8: Rotate Array

Problem statement

Rotate `nums` to the **right** by `k` steps, in place. `k` can be larger than the length.

Examples

Input	Output	Why
`[1, 2, 3, 4, 5, 6, 7]`, k = 3	`[5, 6, 7, 1, 2, 3, 4]`	last 3 elements move to the front
`[1, 2]`, k = 3	`[2, 1]`	3 % 2 = 1 step

Constraints

- 1 <= n <= 10^5
- 0 <= k <= 10^9

Brute-force approach

Shift the whole array one place right, `k` times. After `k %= n`, that is up to n x n steps. A second option copies each element to `(i + k) % n` in a new array: O(n) time, O(n) space.

```
static void rotateBrute(int[] a, int k) {
    int n = a.length;
    if (n == 0) return;
    k %= n;
    for (int step = 0; step < k; step++) {    // move everything one place right, k times
        int last = a[n - 1];
        for (int i = n - 1; i > 0; i--) a[i] = a[i - 1];
        a[0] = last;
    }
}
```

Optimized Java solution

Three reversals. Reverse the whole array, then reverse the first `k` elements, then reverse the remaining `n - k` elements.

```
static void rotate(int[] a, int k) {
    int n = a.length;
    if (n == 0) return;
    k %= n;                                   // k may be larger than n
    reverse(a, 0, n - 1);                     // whole array
    reverse(a, 0, k - 1);                     // first k elements
    reverse(a, k, n - 1);                     // the rest
}

static void reverse(int[] a, int l, int r) {
    while (l < r) {
        int tmp = a[l]; a[l] = a[r]; a[r] = tmp;
        l++; r--;
    }
}
```

Dry run

Step	Array
start	[1, 2, 3, 4, 5, 6, 7]
reverse(0, 6)	[7, 6, 5, 4, 3, 2, 1]
reverse(0, 2) (first k = 3)	[5, 6, 7, 4, 3, 2, 1]
reverse(3, 6) (the rest)	[5, 6, 7, 1, 2, 3, 4]

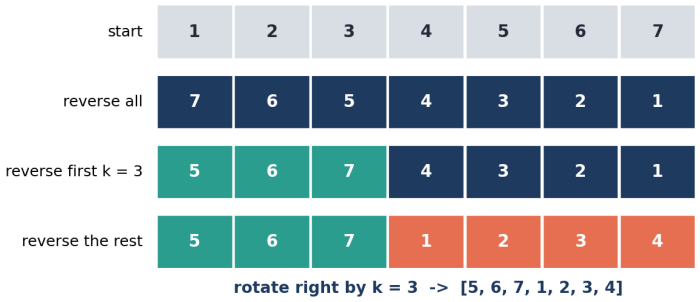


Figure 6.1 Rotate right by 3 using three reversals.

Complexity

Approach	Time	Auxiliary space
Shift one by one, k times	O(n x k) ($O(n^2)$ after k %= n)	O(1)
Extra array	O(n)	O(n)
Three reversals	O(n)	O(1)

Edge cases

- `k = 0` or `k = n`: nothing changes (`k %= n` gives 0, and `reverse(a, 0, -1)` does nothing).
- `k > n`: always reduce with `k %= n`.
- n = 1.
- Rotating **left** by k equals rotating right by n - k.

Explanation

Why does it work? Write the array as two blocks `A B`, where `B` is the last k elements. Rotating right gives `B A`. Reversing everything gives `rev(B) rev(A)`. Reversing each block separately turns `rev(B)` into `B` and `rev(A)` into `A`, giving `B A`. Each element is moved a constant number of times: O(n) time, O(1) space.

6.9 Problem 9: Two Sum

Problem statement

Given an array `nums` and an integer `target`, return the indexes `[i, j]` (with `i < j`) of two numbers that add up to `target`. Exactly one solution exists, and you may not use the same element twice.

Examples

Input	Output	Why
`[2, 7, 11, 15]`, target 9	`[0, 1]`	2 + 7 = 9
`[3, 2, 4]`, target 6	`[1, 2]`	2 + 4 = 6 (not 3 + 3: one 3 only)

Constraints

- 2 <= n <= 10^4
- -10^9 <= nums[i], target <= 10^9
- exactly one valid answer

Brute-force approach

Try every pair with two nested loops.

```
static int[] twoSumBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] + a[j] == target) return new int[]{i, j};
    return new int[]{-1, -1};
}
```

Optimized Java solution

For each number `x`, the partner we need is `target - x`. Store every number we have already passed in a `HashMap` (value to index). If the partner is already in the map, we are done; otherwise add `x` to the map.

```
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> seen = new HashMap<>();     // value -> index
    for (int j = 0; j < a.length; j++) {
        Integer i = seen.get(target - a[j]);          // has the partner appeared?
        if (i != null) return new int[]{i, j};
        seen.put(a[j], j);
    }
    return new int[]{-1, -1};
}
```

Dry run

j	a[j]	need = target - a[j]	map before (value:index)	partner found?
0	3	3	{}	no
1	2	4	{3:0}	no
2	4	2	{3:0, 2:1}	yes -> [1, 2]

Complexity

Approach	Time	Auxiliary space
Two nested loops	O(n^2)	O(1)
HashMap, one pass	O(n)	O(n)

Edge cases

- Same value twice (`[3, 3]`, target 6): works because we look up the partner **before** inserting the current value.
- Negative numbers and zero.
- The answer uses two different indexes, never `i == j`.
- Large values: the sum of two ints can overflow; use `long` if the constraints are bigger than here.

Explanation

We trade memory for speed. Looking up a key in a HashMap is O(1) on average, so the search for a partner no longer needs a loop. Because we scan left to right and store only earlier numbers, the first time a partner exists, the two indexes are `(earlier, current)`.

6.10 Problem 10: Best Time to Buy and Sell Stock

Problem statement

`prices[i]` is the price on day `i`. Buy on one day and sell on a **later** day to maximize profit. Return the maximum profit, or `0` if no profit is possible.

Examples

Input	Output	Why
`[7, 1, 5, 3, 6, 4]`	`5`	buy at 1, sell at 6
`[7, 6, 4, 3, 1]`	`0`	prices only fall

Constraints

- $1 <= n <= 10^5$
- $0 <= prices[i] <= 10^4$

Brute-force approach

Try every buy day and every later sell day.

```
static int maxProfitBrute(int[] p) {
    int best = 0;
    for (int buy = 0; buy < p.length; buy++)
        for (int sell = buy + 1; sell < p.length; sell++)
            best = Math.max(best, p[sell] - p[buy]);
    return best;
}
```

Optimized Java solution

Scan once. Keep the cheapest price seen so far. If we sold today, the best profit would be `price - minPrice`. Keep the largest such profit.

```
static int maxProfit(int[] p) {
    int minPrice = Integer.MAX_VALUE, best = 0;
    for (int price : p) {
        minPrice = Math.min(minPrice, price);         // cheapest day so far
        best = Math.max(best, price - minPrice);      // sell today
    }
    return best;
}
```

Dry run

price	minPrice so far	profit if sold today	best
7	7	0	0
1	1	0	0
5	1	4	4
3	1	2	4
6	1	5	5
4	1	3	5

Complexity

Approach	Time	Auxiliary space
All buy/sell pairs	$O(n^2)$	O(1)
One pass with minPrice	O(n)	O(1)

Edge cases

- Strictly decreasing prices: answer 0 (never trade).
- A single day: answer 0.
- The lowest price comes **after** the highest: you cannot sell before you buy, which is why we track the minimum *so far*.

Explanation

For a fixed sell day, the best buy day is the cheapest day before it. The running minimum gives exactly that in O(1) time. We take the best over all sell days.

6.11 Problem 11: Maximum Subarray (Kadane's Algorithm)

Problem statement

Return the largest sum of any non-empty **contiguous** subarray of `nums`.

Examples

Input	Output	Why
`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`	`6`	subarray [4, -1, 2, 1]
`[-3, -1, -2]`	`-1`	all negative: best is the single element -1

Constraints

- 1 <= n <= 10^5
- -10^4 <= nums[i] <= 10^4

Brute-force approach

For every start `i`, extend the end `j` and keep the running sum. This checks all n(n+1)/2 subarrays (a version that recomputes each sum from scratch would be $O(n^3)$).

```
static int maxSubarrayBrute(int[] a) {
    int best = a[0];
    for (int i = 0; i < a.length; i++) {
        int sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];                    // sum of a[i..j]
            best = Math.max(best, sum);
        }
    }
    return best;
}
```

Optimized Java solution

Let `current` be the best sum of a subarray that **ends at** index `i`. It is either `a[i]` alone (start fresh) or `a[i]` added to the best subarray ending at `i-1`. So `current = max(a[i], current + a[i])`. The answer is the largest `current` ever seen.

```
static int maxSubarray(int[] a) {
    int current = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        current = Math.max(a[i], current + a[i]);     // extend, or start fresh here
        best = Math.max(best, current);
    }
    return best;
}
```

Dry run

i	a[i]	current	best
0	-2	-2	-2
1	1	1	1
2	-3	-2	1
3	4	4	4
4	-1	3	4
5	2	5	5
6	1	6	6
7	-5	1	6
8	4	5	6

Complexity

Approach	Time	Auxiliary space
Two loops with running sum	$O(n^2)$	O(1)
Kadane	O(n)	O(1)

Edge cases

- All negative numbers: initialize with `a[0]`, not 0, otherwise the answer is wrongly 0.
- Single element.
- Zeros and mixed signs.
- Large arrays: the sum could exceed `int`; use `long` if the constraints require.

Explanation

If `current + a[i]` is smaller than `a[i]`, the old sum is negative and only drags us down, so we restart. This single decision per element is why one pass is enough. At index 3 in the dry run, `current` was -2; `max(4, -2 + 4) = 4` restarts at 4. Then 4 + (-1) + 2 + 1 builds up to 6.

6.12 Problem 12: Merge Sorted Arrays

Problem statement

`nums1` has length `m + n`: its first `m` elements are sorted and the last `n` slots are 0 (empty). `nums2` has `n` sorted elements. Merge `nums2` into `nums1` so that `nums1` becomes sorted. Do it in place.

Examples

Input	Output	Why
`nums1 = [1, 3, 5, 0, 0, 0]`, m = 3, `nums2 = [2, 4, 6]`, n = 3	`[1, 2, 3, 4, 5, 6]`	merged

Constraints

- 0 <= m, n <= 200
- both inputs are sorted

Brute-force approach

Copy `nums2` into the empty tail of `nums1` and sort the whole array.

```
static void mergeBrute(int[] a, int m, int[] b, int n) {
    for (int j = 0; j < n; j++) a[m + j] = b[j];      // copy b after the m real items
    Arrays.sort(a);
}
```

Optimized Java solution

Fill `nums1` **from the back**. Three pointers: `i` at the last real element of `nums1`, `j` at the last element of `nums2`, `k` at the last slot. Put the larger of the two candidates at `k`.

```
static void merge(int[] a, int m, int[] b, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;          // write from the back
    while (j >= 0) {
        if (i >= 0 && a[i] > b[j]) a[k--] = a[i--];
        else a[k--] = b[j--];
    }
}
```

Dry run

action	nums1 after
take b[2]=6: a[5] = 6	[1, 3, 5, 0, 0, 6]
a[2]=5 > b[1]=4: a[4] = 5	[1, 3, 5, 0, 5, 6]
take b[1]=4: a[3] = 4	[1, 3, 5, 4, 5, 6]
a[1]=3 > b[0]=2: a[2] = 3	[1, 3, 3, 4, 5, 6]
take b[0]=2: a[1] = 2	[1, 2, 3, 4, 5, 6]

Complexity

Approach	Time	Auxiliary space
Copy + sort	O((m+n) log(m+n))	O(1) or O(log n)
Three pointers from the back	O(m + n)	O(1)

Edge cases

- `n = 0`: the loop does not run; `nums1` is already correct.
- `m = 0`: all of `nums2` is copied (`i` is -1).
- All of `nums2` is smaller than `nums1`: the `nums1` elements slide right.
- Equal values: either order is fine.

Explanation

Filling from the front would overwrite elements of `nums1` we have not used yet. From the back, the free slots are exactly where the largest elements belong, so we never overwrite anything we still need. When `j` reaches -1, the rest of `nums1` is already in place, so no extra loop is needed.

6.13 Problem 13: Majority Element

Problem statement

The **majority element** appears more than `n / 2` times. Return it. You may assume it always exists.

Examples

Input	Output	Why
`[2, 2, 1, 1, 2, 2, 3, 2]`	`2`	2 appears 5 times out of 8
`[3, 3, 4]`	`3`	2 times out of 3

Constraints

- 1 <= n <= 5 x 10^4
- a majority element always exists

Brute-force approach

For every element count how many times it appears. (A HashMap of counts gives O(n) time and O(n) space, a good middle solution.)

```
static int majorityBrute(int[] a) {
    for (int x : a) {
        int count = 0;
        for (int y : a) if (y == x) count++;
        if (count > a.length / 2) return x;
    }
    return -1;
}
```

Optimized Java solution

Boyer-Moore voting. Keep a `candidate` and a `count`. If `count` is 0, take the current number as the new candidate. Add 1 when you see the candidate, subtract 1 otherwise.

```
static int majority(int[] a) {                    // a majority element is guaranteed
    int candidate = 0, count = 0;
    for (int x : a) {
        if (count == 0) candidate = x;            // pick a new candidate
        count += (x == candidate) ? 1 : -1;       // votes for / against
    }
    return candidate;
}
```

Dry run

x	candidate	count
2	2	1
2	2	2
1	2	1
1	2	0
2	2	1
2	2	2
3	2	1
2	2	2

Complexity

Approach	Time	Auxiliary space
Count each element	$O(n^2)$	O(1)
HashMap of counts	O(n)	O(n)
Boyer-Moore voting	O(n)	O(1)

Edge cases

- Single element array.
- Majority element appears only at the end (the candidate may change several times).
- If the problem does **not** guarantee a majority, add a second pass that counts the candidate.

Explanation

Think of pairing each vote for the majority with a vote against it: they cancel. Because the majority has more than half of all votes, it cannot be completely cancelled by the other values, so it survives as the final candidate. In the dry run the candidate changes to 1 and back to 2, but the count always returns to the majority.

6.14 Problem 14: Missing Number

Problem statement

`nums` contains `n` **distinct** numbers taken from the range `0..n`. Exactly one number from that range is missing. Return it.

Examples

Input	Output	Why
`[3, 0, 1]`	`2`	range 0..3, 2 is missing
`[0, 1]`	`2`	range 0..2, 2 is missing
`[9, 6, 4, 2, 3, 5, 7, 0, 1]`	`8`	range 0..9

Constraints

- 1 <= n <= 10^4
- all numbers are distinct

Brute-force approach

For each value `v` from 0 to n, scan the array to see if it is there.

```
static int missingBrute(int[] a) {
    for (int v = 0; v <= a.length; v++) {
        boolean found = false;
        for (int x : a) {
            if (x == v) { found = true; break; }
        }
        if (!found) return v;
    }
    return -1;
}
```

Optimized Java solution

The sum of `0..n` is `n(n+1)/2`. Subtract the actual sum of the array; what remains is the missing number. (Another popular solution XORs all indexes and values: everything cancels except the missing number.)

```
static int missing(int[] a) {
    long n = a.length;
    long expected = n * (n + 1) / 2;              // 0 + 1 + ... + n
    long actual = 0;
    for (int x : a) actual += x;
    return (int) (expected - actual);
}
```

Dry run

Quantity	Value for [3, 0, 1]
n	3
expected = n(n+1)/2	3 x 4 / 2 = 6
actual sum	3 + 0 + 1 = 4
missing = expected - actual	6 - 4 = 2

Complexity

Approach	Time	Auxiliary space
Search for each value	O(n^2)	O(1)
Sum formula (or XOR)	O(n)	O(1)

Edge cases

- The missing number is 0 or n.
- n = 1: `[0]` gives 1 and `[1]` gives 0.
- Overflow: for big n, `n * (n+1)` can exceed `int`, so we use `long`.

Explanation

The formula 1 + 2 + ... + n = n(n+1)/2 is Gauss's trick: pair the first and last numbers, they all add to n + 1, and there are n/2 pairs. A single pass computes the actual sum, so the difference is the missing value.

6.15 Problem 15: Contains Duplicate

Problem statement

Return `true` if any value appears **at least twice** in `nums`, otherwise `false`.

Examples

Input	Output	Why
`[1, 2, 3, 1]`	`true`	1 appears twice
`[1, 2, 3]`	`false`	all distinct

Constraints

- $0 <= n <= 10^5$

Brute-force approach

Compare every pair. Another option: sort a copy (O(n log n), O(n) space) and look for equal neighbours.

```
static boolean hasDuplicateBrute(int[] a) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] == a[j]) return true;
    return false;
}
```

Optimized Java solution

Put each number in a `HashSet`. `add` returns `false` when the number is already in the set: that is a duplicate.

```
static boolean hasDuplicate(int[] a) {
    Set<Integer> seen = new HashSet<>();
    for (int x : a) {
        if (!seen.add(x)) return true;            // add() is false if x was already there
    }
    return false;
}
```

Dry run

x	set before	set.add(x) returns
1	{}	true
2	{1}	true
3	{1, 2}	true
1	{1, 2, 3}	false -> duplicate!

Complexity

Approach	Time	Auxiliary space
All pairs	$O(n^2)$	O(1)
Sort, check neighbours	O(n log n)	O(n) copy, or O(1) if allowed to sort the input
HashSet	O(n)	O(n)

Edge cases

- Empty or single element: false.
- Duplicates far apart (the set handles it; the neighbour method needs sorting first).
- Negative numbers and zero.

Explanation

A set remembers every value seen. Checking membership is O(1) on average, so one pass is enough and we can stop at the first duplicate. The trade-off is O(n) memory.

6.16 Problem 16: Product of Array Except Self

Problem statement

Return an array `res` where `res[i]` is the product of all elements of `nums` except `nums[i]`. **Do not use division.**

Examples

Input	Output	Why
`[1, 2, 3, 4]`	`[24, 12, 8, 6]`	res[0] = 2 x 3 x 4
`[2, 0, 4]`	`[0, 8, 0]`	one zero
`[0, 0, 3]`	`[0, 0, 0]`	two zeros

Constraints

- $2 <= n <= 10^5$
- every product fits in a 32-bit int

Brute-force approach

For each `i`, multiply all the other elements.

```
static int[] productBrute(int[] a) {
    int[] res = new int[a.length];
    for (int i = 0; i < a.length; i++) {
        int prod = 1;
        for (int j = 0; j < a.length; j++) {
            if (j != i) prod *= a[j];
        }
        res[i] = prod;
    }
    return res;
}
```

Optimized Java solution

`res[i]` = (product of everything to the left of `i`) x (product of everything to the right). First pass: store the left products in `res`. Second pass, from the right: multiply `res[i]` by a running `suffix` product.

```
static int[] product(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    res[0] = 1;
    for (int i = 1; i < n; i++) {
```

```
        res[i] = res[i - 1] * a[i - 1];          // product of everything left of i
    }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= suffix;                        // multiply by everything right of i
        suffix *= a[i];
    }
    return res;
}
```

Dry run

After the first (left-to-right) pass, `res = [1, 1, 2, 6]`. Then the right-to-left pass:

i	suffix before	res after
3	1	[1, 1, 2, 6]
2	4	[1, 1, 8, 6]
1	12	[1, 12, 8, 6]
0	24	[24, 12, 8, 6]

Complexity

Approach	Time	Auxiliary space
Product for each index	$O(n^2)$	O(1)*
Prefix x suffix	O(n)	O(1)*

Edge cases

- Zeros: why division fails (division by zero), and why this method works (no division).
- Negative numbers.
- Overflow if products exceed the `int` range (not in these constraints).
- (*) the returned array is the output, so it is not counted as extra space.

Explanation

Division would be the easy idea (total product / `a[i]`), but it breaks when there is a zero. Prefix and suffix products avoid it. We reuse the output array to hold the prefix products, and one variable for the suffix, so the extra space is O(1).

6.17 Problem 17: Maximum Sum of a Window of Size k

Problem statement

Given an array `nums` and an integer `k`, return the maximum sum of any **k consecutive** elements.

Examples

Input	Output	Why
`[2, 1, 5, 1, 3, 2]`, k = 3	`9`	window [5, 1, 3]
`[-4, -1, -3]`, k = 2	`-4`	-1 + -3 = -4 is the best

Constraints

- 1 <= k <= n <= 10^5

Brute-force approach

For every start index, add up the next k elements: (n - k + 1) x k additions.

```
static int maxWindowBrute(int[] a, int k) {
    int best = Integer.MIN_VALUE;
    for (int start = 0; start + k <= a.length; start++) {
```

```
        int sum = 0;
        for (int i = start; i < start + k; i++) sum += a[i];
        best = Math.max(best, sum);
    }
    return best;
}
```

Optimized Java solution

Compute the first window's sum. Then slide by one: add the entering element and subtract the leaving element.

```
static int maxWindow(int[] a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];      // first window
    int best = sum;
    for (int i = k; i < a.length; i++) {
        sum += a[i] - a[i - k];                   // add the new element, drop the old one
        best = Math.max(best, sum);
    }
    return best;
}
```

Dry run

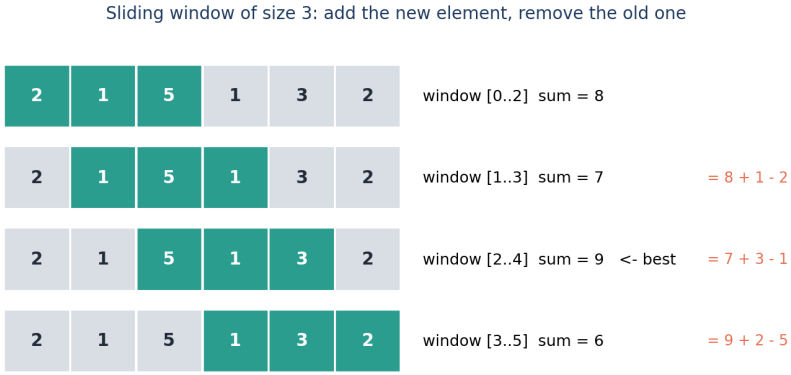


Figure 6.2 Windows of size 3 on [2, 1, 5, 1, 3, 2]. The sums are 8, 7, 9, 6.

Window	Update	Sum	best
[0..2]	2 + 1 + 5	8	8
[1..3]	8 + a[3] - a[0] = 8 + 1 - 2	7	8
[2..4]	7 + a[4] - a[1] = 7 + 3 - 1	9	9
[3..5]	9 + a[5] - a[2] = 9 + 2 - 5	6	9

Complexity

Approach	Time	Auxiliary space
Recompute every window	O(n x k)	O(1)
Sliding window	O(n)	O(1)

Edge cases

- `k = n`: only one window.
- `k = 1`: the answer is the maximum element.
- All negative: do not start `best` at 0 (we start from the first window's sum).
- Always check that `k <= n`.

Explanation

Two neighbouring windows share k - 1 elements. Recomputing them is wasted work. Updating the sum with one addition and one subtraction costs O(1) per slide, so all n - k slides cost O(n).

6.18 Problem 18: Longest Subarray With a Given Sum

Problem statement

Given an integer array `nums` (it may contain **negative numbers**) and a number `target`, return the length of the longest contiguous subarray whose sum equals `target`. Return `0` if none exists.

Examples

Input	Output	Why
`[1, -1, 5, -2, 3]`, target 3	`4`	[1, -1, 5, -2] sums to 3
`[-2, -1, 2, 1]`, target 1	`2`	[-1, 2]
`[1, 2]`, target 10	`0`	no such subarray

Constraints

- 1 <= n <= 10^5
- -10^4 <= nums[i] <= 10^4

Brute-force approach

For each start, extend the end and keep a running sum; record the length whenever it equals the target.

```
static int longestTargetBrute(int[] a, long target) {
    int best = 0;
    for (int i = 0; i < a.length; i++) {
        long sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];
            if (sum == target) best = Math.max(best, j - i + 1);
        }
    }
    return best;
}
```

Optimized Java solution

Let `prefix[i]` be the sum of the first `i+1` elements. A subarray `(s, i]` has sum `prefix[i] - prefix[s]`. We want `prefix[s] = prefix[i] - target`. Store, for every prefix sum, the **earliest** index where it occurred. For each `i`, look up `prefix[i] - target` in the map; the distance `i - start` is a candidate length.

```
static int longestTarget(int[] a, long target) {
    Map<Long, Integer> first = new HashMap<>();   // prefix sum -> earliest index
    first.put(0L, -1);                            // empty prefix ends before index 0
    long sum = 0;
    int best = 0;
    for (int i = 0; i < a.length; i++) {
        sum += a[i];
        Integer start = first.get(sum - target);  // a prefix that leaves exactly 'target'
        if (start != null) best = Math.max(best, i - start);
        first.putIfAbsent(sum, i);                // keep the earliest index only
    }
    return best;
}
```

Dry run

i	a[i]	prefix sum	sum - target	earlier prefix found?	best
0	1	1	-2	none	0
1	-1	0	-3	none	0
2	5	5	2	none	0
3	-2	3	0	start = -1, len 4	4
4	3	6	3	start = 3, len 1	4

Complexity

Approach	Time	Auxiliary space
Two loops	O(n^2)	O(1)
Prefix sums + HashMap	O(n)	O(n)

Edge cases

- Nothing matches: return 0.
- `target = 0`: needs the entry `0 -> -1`, so a prefix that adds to 0 from the very start counts.
- Negative numbers: sliding window does **not** work here, which is why we use the hash map.
- Use `putIfAbsent` so the *earliest* index is kept (earlier start = longer subarray).
- Use `long` for prefix sums.

Explanation

Putting `0 -> -1` in the map represents the empty prefix before index 0, so subarrays that start at index 0 are found. In the dry run at `i = 3` the prefix sum is 3; `3 - 3 = 0` is in the map with index -1, so the length is 3 - (-1) = 4.

6.19 Problem 19: Search in a Rotated Sorted Array

Problem statement

A sorted array of **distinct** values was rotated at an unknown pivot (for example `[0,1,2,4,5,6,7]` became `[4,5,6,7,0,1,2]`). Return the index of `target`, or `-1`. The expected time is O(log n).

Examples

Input	Output	Why
`[4, 5, 6, 7, 0, 1, 2]`, target 0	`4`	found at index 4
`[4, 5, 6, 7, 0, 1, 2]`, target 3	`-1`	not present
`[1]`, target 1	`0`	single element

Constraints

- 1 <= n <= 5000
- all values are distinct

Brute-force approach

Scan the array from left to right. This is O(n) and ignores the sorted structure.

```
static int searchRotatedBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

Optimized Java solution

Binary search still works because **at least one half is always sorted**. Compare `a[lo]` with `a[mid]` to find the sorted half. If the target lies inside the sorted half, search there; otherwise search the other half.

```
static int searchRotated(int[] a, int target) {   // distinct values
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {                     // left half is sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                   // right half is sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Dry run

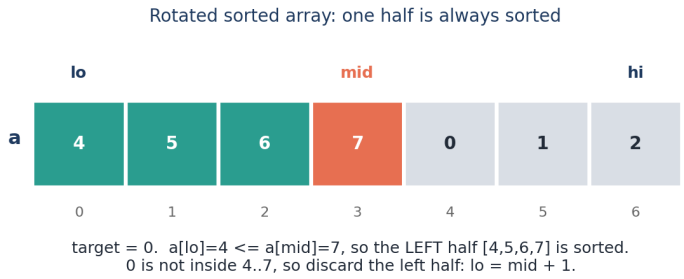


Figure 6.3 First step: the left half [4, 5, 6, 7] is sorted and does not contain 0.

lo	hi	mid	a[mid]	which half is sorted	decision
0	6	3	7	left sorted	not in left: lo = mid + 1
4	6	5	1	left sorted	target in left: hi = mid - 1
4	4	4	0	-	found: return 4

Complexity

Approach	Time	Auxiliary space
Linear scan	O(n)	O(1)
Modified binary search	O(log n)	O(1)

Edge cases

- Array not rotated at all: it is simply sorted, and the same code works.
- One or two elements: `a[lo] <= a[mid]` (with `<=`) handles the case `lo == mid`.
- Target smaller than all or larger than all: returns -1.
- Duplicates would break this solution (it needs distinct values).

Explanation

In a rotated array, the pivot splits it into two sorted runs. Any range `lo..hi` has at least one half without the pivot, and that half is sorted. Checking `a[lo] <= a[mid]` tells us which half is that. Then a normal range test (`a[lo] <= target < a[mid]`) tells us whether the target is in it. Every step halves the range: O(log n).

6.20 Problem 20: Spiral Matrix

Problem statement

Given an `m x n` matrix, return all its elements in **spiral order**: start at the top-left, go right, then down, then left, then up, and keep spiralling inwards.

Examples

Input	Output	Why
`[[1,2,3],[4,5,6],[7,8,9]]`	`[1,2,3,6,9,8,7,4,5]`	3 x 3
`[[1,2,3,4],[5,6,7,8],[9,10,11,12]]`	`[1,2,3,4,8,12,11,10,9,5,6,7]`	3 x 4
`[[1,2],[3,4],[5,6]]`	`[1,2,4,6,5,3]`	3 x 2
`[[7]]`	`[7]`	single cell

Constraints

- 1 <= m, n <= 10

Brute-force approach

Walk step by step, mark each visited cell in a `boolean` matrix, and turn right when the next cell is outside the matrix or already visited.

```
static List<Integer> spiralBrute(int[][] m) {
    int rows = m.length, cols = m[0].length;
    boolean[][] seen = new boolean[rows][cols];   // extra O(rows * cols) memory
    int[] dr = {0, 1, 0, -1}, dc = {1, 0, -1, 0}; // right, down, left, up
    List<Integer> res = new ArrayList<>();
    int r = 0, c = 0, d = 0;
    for (int step = 0; step < rows * cols; step++) {
        res.add(m[r][c]);
        seen[r][c] = true;
        int nr = r + dr[d], nc = c + dc[d];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || seen[nr][nc]) {
            d = (d + 1) % 4;                      // turn right
            nr = r + dr[d];
            nc = c + dc[d];
        }
        r = nr;
        c = nc;
    }
    return res;
}
```

Optimized Java solution

Keep four borders: `top`, `bottom`, `left`, `right`. Walk the top row, the right column, the bottom row and the left column, shrinking the border after each side. The two `if` checks stop us from walking a row or column twice when only one is left.

```
static List<Integer> spiral(int[][] m) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) res.add(m[top][c]);        // top row
        top++;
        for (int r = top; r <= bottom; r++) res.add(m[r][right]);      // right column
        right--;
        if (top <= bottom) {
            for (int c = right; c >= left; c--) res.add(m[bottom][c]); // bottom row
            bottom--;
        }
        if (left <= right) {
            for (int r = bottom; r >= top; r--) res.add(m[r][left]);   // left column
            left++;
        }
    }
```

```
    return res;
}
```

Dry run

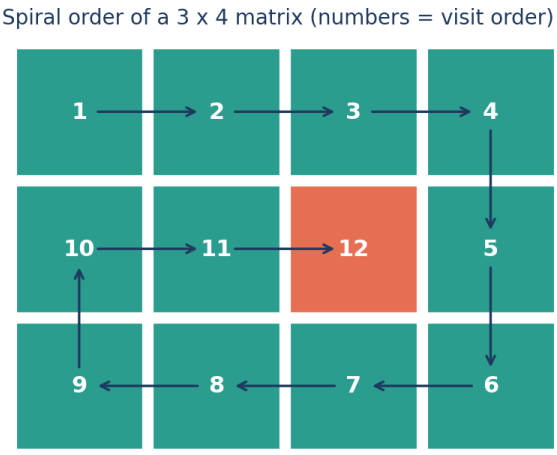


Figure 6.4 The numbers show the visit order for a 3 x 4 matrix.

Round	Side	Cells visited (values)	Border after
1	top row	1, 2, 3, 4	top = 1
1	right column	8, 12	right = 2
1	bottom row	11, 10, 9	bottom = 1
1	left column	5	left = 1
2	top row	6, 7	top = 2
2	right column	none (top > bottom)	right = 1
	end	top > bottom stops the loop	

Complexity

Approach	Time	Auxiliary space
Visited matrix + turn right	O(m x n)	O(m x n)
Shrinking borders	O(m x n)	O(1)*

Edge cases

- Single row: after the top row, `top > bottom` so the other sides are skipped.
- Single column: the check `top <= bottom` stops the bottom row walking back over visited cells.
- Non-square matrices (3 x 4, 3 x 2).
- 1 x 1 matrix.
- (*) the result list is the required output.

Explanation

Every cell is added exactly once, so the time is O(m x n), the minimum possible. The border version needs no extra matrix, only four integers. The two guard conditions are the usual source of bugs: without them, a matrix with one remaining row or column would be traversed twice.

7. 30 Multiple-Choice Questions

Choose the best answer for each question. Questions 1-10 are **beginner**, 11-20 are **intermediate** and 21-30 are **interview level**. The answer key with explanations is in the last chapter. Try to solve every question on paper first.

7.1 Beginner level (Questions 1-10)

Q1. An array has 8 elements. What is the index of its last element?

- **A.** 8
- **B.** 7
- **C.** 9
- **D.** 0

Q2. What does `new int[4]` contain right after it is created?

- **A.** four zeros
- **B.** four random numbers
- **C.** four nulls
- **D.** nothing; you must fill it first

Q3. What does this code print?

```
int[] a = {4, 8, 15};
System.out.println(a.length);
```

- **A.** 2
- **B.** 4
- **C.** 3
- **D.** 15

Q4. What happens when this code runs?

```
int[] a = new int[3];
a[3] = 1;
```

- **A.** a[3] becomes 1 and the array grows
- **B.** ArrayIndexOutOfBoundsException
- **C.** NullPointerException
- **D.** It prints 0

Q5. Which line creates a matrix with 3 rows and 5 columns?

- **A.** int[][] m = new int[5][3];
- **B.** int[][] m = new int[3][5];
- **C.** int[][] m = new int[3, 5];
- **D.** int[][] m = new int[15];

Q6. How long does reading `a[i]` take?

- **A.** O(n)
- **B.** O(log n)
- **C.** O(1)
- **D.** O(i)

Q7. What does this code print?

```
int[] a = {5, 10, 15};
a[1] = a[0] + a[2];
System.out.println(a[1]);
```

A. 20
B. 10
C. 15
D. 30

Q8. Which loop prints the elements of `a` from last to first, with no mistakes?

A. for (int i = a.length; i >= 0; i--)
B. for (int i = a.length - 1; i > 0; i--)
C. for (int i = a.length - 1; i >= 0; i--)
D. for (int i = 0; i < a.length; i--)

Q9. In the worst case, how many comparisons does linear search make on an array of n elements?

A. 1
B. n
C. n / 2
D. log2 n

Q10. What is the default value of each slot in `new String[2]`?

A. "" (the empty string)
B. null
C. 0
D. the text "null"

7.2 Intermediate level (Questions 11-20)

Q11. Inserting a value at index 0 of an array that holds n elements (and has spare room) costs:

A. O(n)
B. O(1)
C. O(log n)
D. O(n^2)

Q12. Binary search requires the array to be:

A. of even length
B. free of duplicates
C. sorted
D. stored in a HashMap

Q13. What does this code print?

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;
System.out.println(a[0]);
```

A. 99
B. 1
C. 0
D. It does not compile

Q14. After rotating `[1, 2, 3, 4, 5]` to the right by 2 steps, the array is:

A. [3, 4, 5, 1, 2]
B. [2, 3, 4, 5, 1]
C. [5, 4, 3, 2, 1]
D. [4, 5, 1, 2, 3]

Q15. Let `prefix[i]` be the sum of the first i elements of `[3, 1, 4, 1, 5]`. What is `prefix[4]`?

A. 8
B. 13
C. 9
D. 14

Q16. A fixed-size sliding window of size k moves over an array of n elements. The total time is:

A. O(n x k)
B. O(n)
C. O(k)
D. O(n^2)

Q17. What does Kadane's algorithm return for `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`?

A. 1
B. 5
C. 7
D. 6

Q18. Two Sum solved with a HashMap has what time and auxiliary space?

A. O(n^2) time and O(1) space
B. O(n) time and O(n) space
C. O(n log n) time and O(1) space
D. O(1) time and O(n) space

Q19. `removeDuplicates` is run on the sorted array `[1, 1, 2, 2, 3]`. What does it return?

A. 2
B. 3
C. 4
D. 5

Q20. The transpose of a 2 x 3 matrix has size:

A. 2 rows x 3 columns
B. 3 rows x 3 columns
C. 6 rows x 1 column
D. 3 rows x 2 columns

7.3 Interview level (Questions 21-30)

Q21. Rotating an array to the right by k with three reversals takes:

A. O(n) time and O(1) extra space
B. O(n x k) time and O(1) extra space
C. O(n) time and O(n) extra space
D. O(k) time and O(1) extra space

Q22. What does this code print?

```
int s = 0;
for (int i = 0; i < 5; i++)
    for (int j = i; j < 5; j++)
        s++;
System.out.println(s);
```

A. 15
B. 10
C. 20
D. 25

Q23. What does Boyer-Moore voting return for `[2, 2, 1, 1, 1, 2, 2]`?

A. 1
B. 0
C. It cannot be decided
D. 2

Q24. For `[1, 2, 3, 4]`, what is the value at index 2 of the product-except-self array?

A. 6
B. 12
C. 8
D. 24

Q25. What is the length of the longest subarray with sum 0 in `[15, -2, 2, -8, 1, 7, 10, 23]`?

A. 3
B. 4
C. 8
D. 5

Q26. Searching for 0 in `[4, 5, 6, 7, 0, 1, 2]`: with lo = 0 and hi = 6, what is `a[mid]` in the first step?

A. 7
B. 5
C. 0
D. 1

Q27. Why is `mid = lo + (hi - lo) / 2` preferred over `mid = (lo + hi) / 2`?

A. lo + hi can overflow int for very large arrays
B. It is faster
C. It gives a different middle for even lengths
D. It also works on unsorted arrays

Q28. What is the spiral order of `[[1, 2], [3, 4]]`?

A. [1, 2, 3, 4]
B. [1, 3, 4, 2]
C. [1, 2, 4, 3]
D. [1, 4, 2, 3]

Q29. What is the maximum sum of 3 consecutive elements of `[2, 1, 5, 1, 3, 2]`?

A. 8
B. 7
C. 11
D. 9

Q30. When merging a sorted `nums2` into `nums1` in place (Problem 12), why do we fill `nums1` from the back?

A. The back of an array is faster to access
B. Sorted arrays must be read from the back
C. It removes the need for any comparison
D. Writing from the front would overwrite elements of nums1 that are not merged yet

8. Coding Practice: 10 Beginner, 10 Intermediate, 5 Advanced

Write each solution yourself before you open the answer key. Every problem lists the target complexity. All the sample outputs in the answer key were produced by running the code.

8.1 Beginner problems (B1-B10)

B1. Sum of an array

Return the sum of all elements of `int[] a`.

Example: `[4, 7, 1, 9, 3]` gives `24`

~~Target: O(n) time, O(1) space; use `long` for the total.~~

B2. Count even numbers

Return how many elements are even.

Example: `[3, 8, 12, 5, 6]` gives `3`

Target: O(n), O(1)

B3. Index of the minimum

Return the index of the smallest element (the first one if it repeats).

Example: `[5, 3, 9, 3, 7]` gives `1`

Target: O(n), O(1)

B4. Palindrome array

Return `true` if the array reads the same forwards and backwards.

Example: `[1, 2, 3, 2, 1]` gives `true`; `[1, 2, 3]` gives `false`

Target: O(n), O(1) with two pointers

B5. Count occurrences

Return how many times `x` appears in the array.

Example: `[2, 5, 2, 2, 7]`, x = 2 gives `3`

Target: O(n), O(1)

B6. Sum at even indices

Return the sum of the elements at indices 0, 2, 4, ...

Example: `[10, 20, 30, 40, 50]` gives `90`

Target: O(n), O(1)

B7. Count above average

Return how many elements are strictly greater than the average. Avoid decimals.

Example: `[1, 2, 3, 4, 10]` gives `1` (average 4, only 10 is greater)

Target: O(n), O(1)

B8. All elements equal

Return `true` if all elements are the same.

Example: `[4, 4, 4]` gives `true`; `[4, 4, 5]` gives `false`

Target: O(n), O(1)

B9. Insert into a new array

Return a new array with `value` inserted at `index`.

Example: `[1, 2, 4, 5]`, index 2, value 3 gives `[1, 2, 3, 4, 5]`

Target: O(n), O(n) for the new array

B10. Delete from a new array

Return a new array with the element at `index` removed.

Example: `[1, 2, 9, 3, 4]`, index 2 gives `[1, 2, 3, 4]`

Target: O(n), O(n) for the new array

8.2 Intermediate problems (I1-I10)

I1. First non-repeating element

Return the first element that appears exactly once, or `null`.

Example: `[4, 5, 1, 2, 0, 4, 5, 1]` gives `2`

Target: O(n), O(n) with a HashMap

I2. Count pairs with a given sum

Count pairs `i < j` with `a[i] + a[j] == k`.

Example: `[1, 5, 7, -1, 5]`, k = 6 gives `3`

Target: O(n), O(n)

I3. Leaders

An element is a leader if it is strictly greater than every element to its right. Return the leaders from left to right.

Example: `[16, 17, 4, 3, 5, 2]` gives `[17, 5, 2]`

Target: O(n), O(1) extra (scan from the right)

I4. Equilibrium index

Return the first index where the sum of the elements on its left equals the sum on its right (the element itself is excluded), or -1.

Example: `[-7, 1, 5, 2, -4, 3, 0]` gives `3`

Target: O(n), O(1)

I5. Intersection of two arrays

Return the sorted distinct values that appear in both arrays.

Example: `[1, 2, 2, 1]` and `[2, 2, 3]` gives `[2]`

Target: O(n + m) average, O(n) space

I6. Sort 0s, 1s and 2s

Sort an array that contains only 0, 1 and 2 in place, in a single pass, without `Arrays.sort`.

Example: `[2, 0, 2, 1, 1, 0]` gives `[0, 0, 1, 1, 2, 2]`

Target: O(n), O(1) (Dutch national flag)

I7. Maximum consecutive ones

Return the longest run of consecutive 1s in a 0/1 array.

Example: `[1, 1, 0, 1, 1, 1]` gives `3`

Target: O(n), O(1)

I8. Count subarrays with sum k

Count the contiguous subarrays whose sum is exactly `k` (negative numbers allowed).

Example: `[3, 4, 7, 2, -3, 1, 4, 2]`, k = 7 gives `4`

Target: O(n), O(n) with prefix sums and a HashMap

I9. Container with most water

`h[i]` are the heights of vertical lines. Choose two lines that, with the x-axis, hold the most water. Return the maximum area = min(height) x distance.

Example: `[1, 8, 6, 2, 5, 4, 8, 3, 7]` gives `49`

Target: O(n), O(1) with two pointers

I10. Diagonal sum

For a square matrix return the sum of both diagonals; count the centre cell once.

Example: `[[1,2,3],[4,5,6],[7,8,9]]` gives `25`

Target: O(n), O(1)

8.3 Advanced problems (A1-A5)

A1. Trapping rain water

`h[i]` is the height of a bar of width 1. After rain, how many units of water are trapped between the bars?

Example: `[0,1,0,2,1,0,1,3,2,1,2,1]` gives `6`

Target: O(n) time, O(1) space with two pointers

A2. Longest consecutive sequence

Return the length of the longest run of consecutive integers (any order in the array, e.g. 1, 2, 3, 4).

Example: `[100, 4, 200, 1, 3, 2]` gives `4`

Target: O(n) average, O(n) space with a HashSet

A3. First missing positive

Return the smallest positive integer that does not appear in the array.

Example: `[3, 4, -1, 1]` gives `2`; `[1, 2, 0]` gives `3`; `[7, 8, 9]` gives `1`

Target: O(n) time, O(1) extra space (you may modify the array)

A4. Sliding window maximum

Return the maximum of every window of size `k`.

Example: `[1, 3, -1, -3, 5, 3, 6, 7]`, k = 3 gives `[3, 3, 5, 5, 6, 7]`

Target: O(n) time, O(k) space with a deque

A5. 3Sum

Return all unique triplets that add up to 0.

Example: `[-1, 0, 1, 2, -1, -4]` gives `[[-1, -1, 2], [-1, 0, 1]]`

Target: O(n^2) time; sort + two pointers; skip duplicates

9. Interview Questions and Answers

> **How to answer an array question (7 steps)**
>
> **1. Repeat** the problem in your own words. **2. Ask** about constraints: size of n, negative numbers, duplicates, sorted or not, can I modify the input?
>
> **3. Try a small example** by hand. **4. Say the brute force** and its complexity. **5. Improve it**: which pattern fits (two pointers, window, prefix, hash, binary search)?
>
> **6. Code** cleanly with good names. **7. Test** with edge cases, then state time and **auxiliary** space.

Q1. What is an array and what are its limits?

An array is a fixed-size block of same-type elements stored side by side, accessed by index in O(1). Its size cannot change after creation, and inserting or deleting in the middle needs O(n) shifting.

Q2. Array vs `ArrayList`?

An array has a fixed size and can hold primitives (`int[]`). `ArrayList` grows automatically but holds objects (`Integer`, not `int`). `ArrayList.add` at the end is amortized O(1); `add(0, x)` is O(n). Arrays are lighter and faster for primitives.

Q3. Why is reading `a[i]` O(1)?

Elements are contiguous, so the address is `start + i x elementSize`. No searching is needed.

Q4. Find the second largest element without sorting.

Keep two variables, `first` and `second`. Update them in one pass (Problem 2). O(n) time, O(1) space. Handle duplicates of the maximum and arrays with fewer than two distinct values.

Q5. How do you reverse an array in place? Complexity?

Two pointers swap the ends and move inward (Problem 4). O(n) time, O(1) space.

Q6. How do you detect duplicates? Compare the options.

Nested loops: $O(n^2)$ time, O(1) space. Sort then compare neighbours: O(n log n) time. HashSet: O(n) time, O(n) space. Choose by the constraints and whether you may modify the input.

Q7. Solve Two Sum. Why do we look up before inserting?

For each number look up `target - x` in a HashMap of earlier numbers; otherwise store `x` (Problem 9). Looking up first prevents using the same element twice and handles `[3, 3]` with target 6 correctly.

Q8. Two pointers vs sliding window?

Two pointers usually start at the two ends of a **sorted** array and move inward, or one reads while one writes. A sliding window is a **contiguous range** `[left, right]` that expands on the right and shrinks on the left. Both are O(n) because each pointer only moves forward.

Q9. What are prefix sums and when do you use them?

`prefix[i]` stores the sum of the first i elements, so a range sum is `prefix[r+1] - prefix[l]` in O(1). Use them when there are many range queries, or when a problem says "sum of subarray" (equilibrium index, subarray sum equals k).

Q10. Explain Kadane's algorithm in one minute.

For each index keep `current`, the best sum of a subarray ending here: `current = max(a[i], current + a[i])`. If the old sum is negative we restart. Track the maximum `current`. O(n) time, O(1) space. Initialize with `a[0]` so all-negative arrays work.

Q11. Why must binary search use a sorted array, and why `lo + (hi - lo) / 2`?

Each step discards half the range based on the order of the data. `(lo + hi) / 2` can overflow `int` for very large indexes; `lo + (hi - lo) / 2` cannot.

Q12. How do you search a rotated sorted array in O(log n)?

At every step one half is sorted. Compare `a[lo]` with `a[mid]` to find it, check whether the target lies inside that half, and discard the other half (Problem 19).

Q13. What does "in place" mean?

The algorithm changes the input array itself and uses only O(1) extra memory (for example reverse, move zeroes, remove duplicates). The result array you must return is usually not counted as extra space, but say so.

Q14. Rotate an array by k in O(n) time and O(1) space.

Reverse the whole array, then the first k elements, then the rest. First reduce `k %= n` (Problem 8).

Q15. What does "amortized O(1)" mean for `ArrayList.add`?

Most adds are O(1). Occasionally the array is full and all elements are copied to a bigger array, which costs O(n). Spread over many adds, the average cost per add is still a constant.

Q16. `==` versus `Arrays.equals`, and what is an alias?

`==` compares references (are they the same object?). `Arrays.equals` compares contents. `int[] b = a;` makes an alias: both names refer to one array, so changes through `b` appear in `a`. Use `Arrays.copyOf` for a real copy.

Q17. How do you avoid integer overflow when summing?

Use `long` for sums and products if values can be large, and cast **before** adding: `(long) a + b`. Also compute the midpoint safely in binary search.

Q18. This code has a `while` loop inside a `for` loop. Is it O(n^2)?

Not necessarily. In a sliding window, `right` moves n times and `left` moves at most n times in total, so the work is at most 2n, which is O(n). Count how often each variable moves, not how the loops are nested.

Q19. The input has n up to 10^5. What complexity should you aim for?

About 10^8 simple operations fit in one second, so O(n^2) = 10^{10} is too slow. Aim for O(n) or O(n log n). If n is at most 20, exponential solutions can be fine.

Q20. Which edge cases do you test?

Empty array, one element, two elements, all equal, sorted and reverse-sorted, negatives and zeros, duplicates, very large values, `k = 0` or `k > n`, target missing or at either end, and for matrices: one row, one column, non-square (see the checklist).

10. Common Java Mistakes with Arrays

Every example below was compiled and run. The comment shows what you will see.

10.1 Off-by-one: going past the last index

```
static int sumWrong(int[] a) {
    int s = 0;
    for (int i = 0; i <= a.length; i++) {     // BUG: i reaches a.length (one step too far)
        s += a[i];
    }
    return s;
}

static int sumRight(int[] a) {
    int s = 0;
    for (int i = 0; i < a.length; i++) {      // FIX: stop at length - 1
        s += a[i];
    }
    return s;
}
```

```
Index 3 out of bounds for length 3
6
```

10.2 Thinking `b = a` makes a copy

```
static void copyDemo() {
    int[] a = {1, 2, 3};
    int[] wrong = a;                          // BUG: not a copy, same array
    wrong[0] = 99;
    System.out.println(a[0]);                 // 99

    int[] b = {1, 2, 3};
    int[] right = Arrays.copyOf(b, b.length); // FIX: real copy
    right[0] = 99;
    System.out.println(b[0]);                 // 1
}
```

```
99
1
```

10.3 Comparing arrays with `==`

```
static void compareDemo() {
    int[] x = {1, 2, 3};
    int[] y = {1, 2, 3};
    System.out.println(x == y);               // false: different objects
    System.out.println(Arrays.equals(x, y));  // true: same contents
}
```

```
false
true
```

10.4 Changing the loop variable of a for-each

```
static void forEachDemo() {
    int[] a = {1, 2, 3};
    for (int x : a) {
        x = x * 10;                           // changes only the copy x
    }
    System.out.println(Arrays.toString(a));   // [1, 2, 3]
    for (int i = 0; i < a.length; i++) {
        a[i] = a[i] * 10;                     // use the index to change the array
    }
    System.out.println(Arrays.toString(a));   // [10, 20, 30]
}
```

```
[1, 2, 3]
[10, 20, 30]
```

10.5 Integer overflow in sums

```
static void overflowDemo() {
    int a = 2_000_000_000, b = 2_000_000_000;
    int wrong = a + b;                          // BUG: int overflow
    long right = (long) a + b;                  // FIX: widen before adding
    System.out.println(wrong + " " + right);
}
```

```
-294967296 4000000000
```

10.6 Integer division when computing an average

```
static void averageDemo() {
    int[] a = {1, 2};
    int sum = a[0] + a[1];
    System.out.println(sum / a.length);              // 1   (integer division)
    System.out.println((double) sum / a.length);     // 1.5
}
```

```
1
1.5
```

10.7 `Arrays.asList` with a primitive array

```
static void asListDemo() {
    int[] prim = {1, 2, 3};
    Integer[] boxed = {1, 2, 3};
    System.out.println(Arrays.asList(prim).size());   // 1  (a list holding ONE int[] object)
    System.out.println(Arrays.asList(boxed).size());  // 3
}
```

```
1
3
```

10.8 Using a slot of an object array before filling it

```
static void nullDemo() {
    String[] names = new String[2];           // both slots are null
    try {
        System.out.println(names[0].length());
    } catch (NullPointerException e) {
        System.out.println("NullPointerException");
    }
    names[0] = "Asha";                        // fill the slot first
    System.out.println(names[0].length());
}
```

```
NullPointerException
4
```

10.9 Mixing up rows and columns

```
static void matrixDemo() {
    int[][] m = new int[2][3];                // 2 rows, 3 columns
    for (int r = 0; r < m.length; r++)
        for (int c = 0; c < m.length; c++)    // BUG: columns are m[0].length, not m.length
            m[r][c] = 1;
    System.out.println(Arrays.deepToString(m));   // third column was never filled

    int[][] g = new int[2][3];
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++) // FIX: columns come from g[0].length
            g[r][c] = 1;
    System.out.println(Arrays.deepToString(g));
}
```

```
[[1, 1, 0], [1, 1, 0]]
[[1, 1, 1], [1, 1, 1]]
```

10.10 Sorting an `int[]` in descending order

`Arrays.sort(intArray, Collections.reverseOrder())` does **not compile**, because the comparator version needs an object array. Use `Integer[]`, or sort ascending and then reverse.

```
static void sortDescDemo() {
    Integer[] a = {3, 1, 2};                  // must be Integer[], not int[]
    Arrays.sort(a, Collections.reverseOrder());
    System.out.println(Arrays.toString(a));   // [3, 2, 1]
}
```

```
[3, 2, 1]
```

Other common slips	How to avoid them
Starting `max` at 0	Start with `a[0]` (or `Integer.MIN_VALUE`)
Using `-1` as "not found" when -1 can be a real answer	Return `null` / a boolean, or document it clearly
Modifying an array while searching it	Work on a copy (`clone()`) or say that you change the input
Forgetting that `String.length()` has brackets but `array.length` does not	Arrays: `a.length`; Strings: `s.length()`; lists: `list.size()`
Nested loop that starts at `j = i` when pairs must be different	Use `j = i + 1`

11. Edge-Case Checklist

Before you say "done", run through this list on your code (in your head or on paper).

Situation	Questions to ask	Typical bug
Empty array	Does the code touch `a[0]`?	Exception, or wrong return value
One element	Does the loop run zero times correctly?	Wrong initial value
Two elements	Do pointers cross correctly?	Skips or repeats a pair
All elements equal	Duplicates, ties, `<` vs `<=`	Wrong second largest, wrong k
Already sorted / reverse sorted	Best and worst case of the algorithm	Unexpected $O(n^2)$
Negative numbers and zeros	Is the start value 0? Is division involved?	Max sum 0 for all-negative arrays
Duplicates	First or last index? Count each once?	Double counting
Large values	Can the sum or product overflow `int`?	Negative sum from big positives
`k = 0`, `k = n`, `k > n`	Use `k %= n`; check `k <= n` for windows	Division by zero, out of range
Target missing / first / last	What is returned when nothing is found?	Off-by-one at the ends
Even vs odd length	Where do the pointers meet?	Middle element handled twice
Matrix: single row, single column, non-square	Do loops use `m[0].length` for columns?	Spiral repeats cells, rows/cols swapped
Input modified?	Is changing the input allowed?	Caller's array is destroyed

12. Complexity Tables

12.1 The twenty problems

#	Problem	Brute force	Optimized	Aux space
1	Maximum	$O(n^2)$	O(n)	O(1)
2	Second largest	O(n log n) sort	O(n)	O(1)
3	Linear search	O(n)	O(n); O(log n) if sorted	O(1)
4	Reverse	O(n), O(n) space	O(n)	O(1)
5	Sorted check	$O(n^2)$	O(n)	O(1)
6	Remove duplicates	$O(n^2)$	O(n)	O(1)
7	Move zeroes	O(n), O(n) space	O(n)	O(1)
8	Rotate array	O(n x k)	O(n)	O(1)
9	Two Sum	$O(n^2)$	O(n)	O(n)
10	Stock profit	$O(n^2)$	O(n)	O(1)
11	Kadane	$O(n^2)$	O(n)	O(1)
12	Merge sorted arrays	O((m+n) log(m+n))	O(m + n)	O(1)
13	Majority element	$O(n^2)$	O(n)	O(1)
14	Missing number	$O(n^2)$	O(n)	O(1)
15	Contains duplicate	$O(n^2)$	O(n)	O(n)
16	Product except self	$O(n^2)$	O(n)	O(1)*
17	Max sum window	O(n x k)	O(n)	O(1)
18	Longest subarray, sum = target	$O(n^2)$	O(n)	O(n)
19	Rotated array search	O(n)	O(log n)	O(1)
20	Spiral matrix	O(r x c), O(r x c) space	O(r x c)	O(1)*

(*) not counting the output.

12.2 Java collections and arrays

Operation	Array `int[]`	`ArrayList`	`HashMap` / `HashSet`	`TreeMap` / `TreeSet`
Get by index	O(1)	O(1)	-	-
Search by value	O(n) (O(log n) if sorted)	O(n)	O(1) average	O(log n)
Add at end	- (fixed size)	O(1) amortized	O(1) average	O(log n)
Insert / remove at index 0	O(n)	O(n)	-	-
Iterate all	O(n)	O(n)	O(n)	O(n), in sorted order
Memory	compact	boxed objects	extra table	tree nodes

12.3 Pattern summary

Pattern	Typical time	Typical extra space	Key line
Two pointers	O(n)	O(1)	`while (left < right)`
Sliding window	O(n)	O(1)	`sum += a[i] - a[i - k]`
Prefix sums	O(n) build, O(1) query	O(n)	`prefix[r+1] - prefix[l]`
Hashing	O(n)	O(n)	`map.get(target - x)`
Kadane	O(n)	O(1)	`cur = Math.max(a[i], cur + a[i])`
Binary search	O(log n)	O(1)	`mid = lo + (hi - lo) / 2`

13. One-Page Revision Sheet

Array facts	Rules
Index 0 .. length-1; `a.length` (no brackets); default 0 / false / null	`b = a` is an alias; copy with `Arrays.copyOf`; compare with `Arrays.equals`
Fixed size; access O(1); insert/delete at index O(n)	Use `long` for big sums; `lo + (hi - lo) / 2`; start max with `a[0]`

Pattern	Use when	Skeleton	Time / space
Two pointers	sorted, pair, in-place, palindrome	`l=0, r=n-1; while (l<r) ...`	O(n) / O(1)
Sliding window	subarray of size k, longest/shortest with condition	`sum += a[r]; while (bad) sum -= a[l++]`	O(n) / O(1)
Prefix sum	range sums, equilibrium, subarray = k	`p[i+1] = p[i] + a[i]`; range = `p[r+1]-p[l]`	O(n) / O(n)
Hashing	seen before? count? partner value?	`map.get(t - x)`, `set.add(x)`	O(n) / O(n)
Kadane	max contiguous sum	`cur = max(a[i], cur + a[i])`	O(n) / O(1)
Binary search	sorted or rotated sorted	`while (lo<=hi) { mid = lo+(hi-lo)/2 ... }`	O(log n) / O(1)

Formulas	Complexity ladder	Common edge cases
1+2+...+n = n(n+1)/2	O(1) < O(log n) < O(n) < O(n log n) < $O(n^2)$ < $O(n^3)$ < $O(2^n)$	empty, one element, two elements
(n-1)+...+1 = n(n-1)/2 (pairs)	binary search: floor(log2 n) + 1 steps	all equal, sorted, reverse sorted
rotate right: reverse all, first k, rest; `k %= n`	n = 10^5: need O(n) or O(n log n)	negatives, zeros, overflow, k = 0 or k > n
rotate matrix 90 clockwise: transpose + reverse rows	10^8 simple ops per second (estimate)	target missing / first / last; matrix 1 row, 1 column

Problem to idea		Problem to idea	
Maximum / second largest	one pass, 1-2 variables	Contains duplicate	HashSet
Reverse / palindrome	two pointers	Two Sum	HashMap: value to index, look up before insert
Remove duplicates, move zeroes	read and write pointers	Product except self	prefix x suffix, no division
Rotate array	three reversals	Max sum window k	slide: add new, subtract old
Merge sorted arrays	fill from the back	Longest subarray sum = target	prefix sums + earliest index
Stock profit	track min so far	Rotated array search	find the sorted half
Kadane	extend or restart	Spiral matrix	4 borders, guard the last row/column
Majority element	Boyer-Moore voting	Missing number	n(n+1)/2 minus the sum, or XOR

Before you submit	
1. Did I handle empty / single element? 2. Off-by-one in loops (< vs <=)? 3. Can anything overflow? 4. Did I say time AND auxiliary space?	5. Did I dry-run one example? 6. Is the input allowed to be modified? 7. Is -1 a real answer? 8. Rows vs columns in 2D?

14. 7-Day Study Plan

Plan 2 to 3 hours per day. For every problem: try for 20 minutes alone, then read the explanation, then **close the book and code it again** from memory the next day. Keep a notebook of mistakes.

Day	Learn	Practise	Goal at the end of the day
1	Chapters 1-2: arrays, indexing, input/output, traversal, complexity of loops	Problems 1, 2, 3, 5; B1-B8; MCQ 1-10	You can write loops without off-by-one errors and explain O(n) vs $O(n^2)$.
2	Chapter 3: insertion and deletion; in-place ideas	Problems 4, 6, 7; B9, B10; I7	You can explain read/write pointers.
3	Two pointers and rotation (Pattern 1)	Problems 8, 12; I3, I6, I9	You can solve sorted-array problems in O(n) and O(1) space.
4	Hashing and the prefix idea (Patterns 3 and 4)	Problems 9, 13, 14, 15; I1, I2, I5	You can say when a HashMap or HashSet is worth O(n) space.
5	Sliding window, Kadane, prefix sums	Problems 10, 11, 16, 17, 18; I4, I8; MCQ 11-20	You can recognise window and prefix problems from the wording.
6	Chapter 5 (matrices) and binary search	Problems 19, 20; I10; A3; MCQ 21-30	You can rotate, transpose and spiral a matrix without bugs.
7	Advanced mixes and revision	A1, A2, A4, A5; revision sheet; one timed mock: 2 problems in 45 minutes	You can explain brute force, optimization, code, dry run and complexity out loud.

Daily routine

10 min: review yesterday's notes. **40 min**: learn one pattern. **90 min**: practise problems. **10 min**: write down the mistakes and the one trick of the day.

After day 7, repeat the problems you could not finish in 20 minutes. Spaced repetition beats cramming.

Interview tips

Think out loud. State the brute force first, even if it is slow. Mention time **and** auxiliary space. Test with a small example and one edge case. If you are stuck, ask a question about the constraints: it is allowed and often gives the hint.

Reminder: all questions in this book are original practice material. They are not claimed to be actual Google or Microsoft interview questions.

15. Answer Key

15.1 Multiple-choice answers and explanations

Q	Level	Answer	Explanation
1	Beginner	**B** 7	Indexes run from 0 to length - 1, so the last index is 8 - 1 = 7.
2	Beginner	**A** four zeros	Java fills int arrays with the default value 0. (`null` is the default only for object arrays such as `String[]`.)
3	Beginner	**C** 3	`a.length` is the number of elements, which is 3 (the highest index is 2).
4	Beginner	**B** ArrayIndexOutOfBoundsException	Valid indexes are 0, 1, 2. Index 3 is outside the array, so Java throws ArrayIndexOutOfBoundsException.
5	Beginner	**B** int[][] m = new int[3][5];	The first size is the number of rows and the second is the number of columns. `new int[3, 5]` is not valid Java.
6	Beginner	**C** O(1)	Array slots are consecutive in memory, so the address of `a[i]` is computed directly: constant time.
7	Beginner	**A** 20	a[1] becomes 5 + 15 = 20 and then 20 is printed.
8	Beginner	**C** for (int i = a.length - 1; i >= 0; i--)	It starts at the last index and stops after index 0. The first option starts out of range, the second skips index 0, and the last one never ends properly.
9	Beginner	**B** n	If the target is last or missing, every element is compared once: n comparisons (O(n)).
10	Beginner	**B** null	Object arrays are filled with `null`, which means "no object yet".
11	Intermediate	**A** O(n)	All n elements must shift one place to the right, so the cost grows linearly with n.
12	Intermediate	**C** sorted	Binary search discards half the range based on order. Without sorted data the decision is meaningless.
13	Intermediate	**A** 99	`b = a` copies the reference, not the array. Both names refer to the same array, so the change is visible through `a`.
14	Intermediate	**D** [4, 5, 1, 2, 3]	The last two elements (4 and 5) move to the front: [4, 5, 1, 2, 3].
15	Intermediate	**C** 9	prefix[4] = 3 + 1 + 4 + 1 = 9.
16	Intermediate	**B** O(n)	Each slide adds one element and removes one element: O(1). There are n - k slides, so O(n).
17	Intermediate	**D** 6	The best subarray is [4, -1, 2, 1] with sum 6. (The sum of the whole array is only 1.)
18	Intermediate	**B** O(n) time and O(n) space	One pass (O(n)) while storing up to n values in the map (O(n) memory).
19	Intermediate	**B** 3	There are three distinct values: 1, 2, 3, so k = 3.
20	Intermediate	**D** 3 rows x 2 columns	Transpose swaps rows and columns: an r x c matrix becomes c x r.
21	Interview	**A** O(n) time and O(1) extra space	Each element is swapped a constant number of times (O(n)), and only a few index variables are used (O(1) space).
22	Interview	**A** 15	The inner loop runs 5, 4, 3, 2, 1 times: 5 + 4 + 3 + 2 + 1 = 15.
23	Interview	**D** 2	2 appears 4 times out of 7, which is more than half, so the surviving candidate is 2.
24	Interview	**C** 8	res[2] = 1 x 2 x 4 = 8 (everything except the 3).

Q	Level	Answer	Explanation
25	Interview	**D** 5	The subarray [-2, 2, -8, 1, 7] sums to 0 and has length 5.
26	Interview	**A** 7	mid = 0 + (6 - 0) / 2 = 3, and a[3] = 7.
27	Interview	**A** lo + hi can overflow int for very large arrays	If lo and hi are both near 2 billion, lo + hi exceeds the int range and becomes negative. lo + (hi - lo) / 2 never overflows.
28	Interview	**C** [1, 2, 4, 3]	Right along the top row (1, 2), down the right column (4), left along the bottom row (3).
29	Interview	**D** 9	The window sums are 8, 7, 9, 6. The maximum is 9 (from 5 + 1 + 3).
30	Interview	**D** Writing from the front would overwrite elements of nums1 that are not merged yet	The empty slots are at the back. Writing the largest values there never destroys data that we still need.

How the answers were verified: every numeric answer (Q7, Q14, Q15, Q17, Q19, Q22, Q23, Q24, Q25, Q26, Q28, Q29) was reproduced by running Java code; the others follow directly from definitions explained in the chapters.

15.2 Coding practice: solutions

B1

Add every element to a `long` total so large inputs cannot overflow.

```
static long sum(int[] a) {
    long total = 0;
    for (int x : a) total += x;
    return total;
}
```

Verified output: `24` **Complexity:** O(n) time, O(1) space

B2

Test `x % 2 == 0`; this also works for negative numbers.

```
static int countEven(int[] a) {
    int count = 0;
    for (int x : a) {
        if (x % 2 == 0) count++;              // works for negative numbers too: -4 % 2 == 0
    }
    return count;
}
```

Verified output: `3` **Complexity:** O(n), O(1)

B3

Remember the index of the smallest value; the strict `<` keeps the first minimum.

```
static int indexOfMin(int[] a) {
    int minIndex = 0;
    for (int i = 1; i < a.length; i++) {
        if (a[i] < a[minIndex]) minIndex = i;     // strict <, so the FIRST minimum stays
    }
    return minIndex;
}
```

Verified output: `1` **Complexity:** O(n), O(1)

B4

Compare the two ends and move inwards.

```
static boolean isPalindrome(int[] a) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        if (a[l] != a[r]) return false;
        l++;
        r--;
    }
    return true;
}
```

Verified output: `true | false` **Complexity:** O(n), O(1)

B5

Count the matches.

```
static int countOccurrences(int[] a, int x) {
    int count = 0;
    for (int v : a) {
        if (v == x) count++;
    }
    return count;
}
```

Verified output: `3` **Complexity:** O(n), O(1)

B6

Step the index by 2 starting at 0.

```
static long sumEvenIndices(int[] a) {
    long total = 0;
    for (int i = 0; i < a.length; i += 2) {       // 0, 2, 4, ...
        total += a[i];
    }
    return total;
}
```

Verified output: `90` **Complexity:** O(n), O(1)

B7

Compute the total first. `x > total / n` becomes `x * n > total` (use `long`), so no decimals are needed.

```
static int countAboveAverage(int[] a) {
    long total = 0;
    for (int x : a) total += x;
    int count = 0;
    for (int x : a) {
        if ((long) x * a.length > total) count++;   // x > total/n without decimals
    }
    return count;
}
```

Verified output: `1` **Complexity:** O(n), O(1)

B8

Compare every element with the first one.

```
static boolean allEqual(int[] a) {
    for (int i = 1; i < a.length; i++) {
        if (a[i] != a[0]) return false;
    }
    return true;
}
```

Verified output: `true | false` **Complexity:** O(n), O(1)

B9

Copy the part before the index, write the value, copy the rest one place later.

```
static int[] insertNew(int[] a, int index, int value) {
    int[] r = new int[a.length + 1];
    for (int i = 0; i < index; i++) r[i] = a[i];          // before the new slot
    r[index] = value;
    for (int i = index; i < a.length; i++) r[i + 1] = a[i];   // after the new slot
    return r;
}
```

Verified output: `[1, 2, 3, 4, 5]` **Complexity:** O(n) time, O(n) space (new array)

B10

Copy every element except the one at `index`.

```
static int[] deleteNew(int[] a, int index) {
    int[] r = new int[a.length - 1];
    for (int i = 0, k = 0; i < a.length; i++) {
        if (i != index) r[k++] = a[i];                    // skip the deleted index
    }
    return r;
}
```

Verified output: `[1, 2, 3, 4]` **Complexity:** O(n) time, O(n) space (new array)

I1

Count every value in a HashMap, then scan the array again and return the first value with count 1.

```
static Integer firstNonRepeating(int[] a) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : a) freq.merge(x, 1, Integer::sum);       // count every value
    for (int x : a) {
        if (freq.get(x) == 1) return x;                   // first one that appears once
    }
    return null;
}
```

Verified output: `2 | null` **Complexity:** O(n) time, O(n) space

I2

For each `x`, the earlier numbers equal to `k - x` each form a pair with `x`. Add their count, then record `x`.

```
static int countPairs(int[] a, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    int pairs = 0;
    for (int x : a) {
        pairs += freq.getOrDefault(k - x, 0);             // earlier partners of x
        freq.merge(x, 1, Integer::sum);
    }
    return pairs;
}
```

Verified output: `3` **Complexity:** O(n) time, O(n) space

I3

Scan from the right while remembering the largest value seen. A new maximum is a leader. `addFirst` restores left-to-right order.

```
static List<Integer> leaders(int[] a) {
    LinkedList<Integer> res = new LinkedList<>();
    int maxRight = Integer.MIN_VALUE;
    for (int i = a.length - 1; i >= 0; i--) {            // scan from the right
        if (a[i] > maxRight) {
            res.addFirst(a[i]);                          // keep left-to-right order
            maxRight = a[i];
        }
    }
    return res;
}
```

Verified output: `[17, 5, 2]` **Complexity:** O(n) time, O(1) extra space

I4

Total sum first. At index i the right sum is `total - left - a[i]`; compare with `left`.

```
static int equilibrium(int[] a) {
    long total = 0;
    for (int x : a) total += x;
    long left = 0;
    for (int i = 0; i < a.length; i++) {
        long right = total - left - a[i];                // sum to the right of i
        if (left == right) return i;
        left += a[i];
    }
    return -1;
}
```

Verified output: `3 | -1` **Complexity:** O(n) time, O(1) space

I5

Put the first array in a HashSet, collect matches from the second array in a TreeSet (sorted, no duplicates).

```
static int[] intersection(int[] a, int[] b) {
    Set<Integer> inA = new HashSet<>();
    for (int x : a) inA.add(x);
    TreeSet<Integer> common = new TreeSet<>();           // sorted, no duplicates
    for (int x : b) {
        if (inA.contains(x)) common.add(x);
    }
    int[] res = new int[common.size()];
    int i = 0;
    for (int x : common) res[i++] = x;
    return res;
}
```

Verified output: `[2] | [4, 9]` **Complexity:** O(n + m) average, O(n) space

I6

Dutch national flag: `low` marks the end of the 0s, `high` the start of the 2s, `mid` scans. Do not advance `mid` after swapping with `high` because the new value is unchecked.

```
static void sortColors(int[] a) {                          // values are only 0, 1, 2
    int low = 0, mid = 0, high = a.length - 1;
    while (mid <= high) {
        if (a[mid] == 0) {
            int t = a[low]; a[low] = a[mid]; a[mid] = t;
            low++; mid++;
        } else if (a[mid] == 1) {
            mid++;
        } else {
            int t = a[mid]; a[mid] = a[high]; a[high] = t;
            high--;                                        // do not move mid: new value unchecked
        }
    }
}
```

Verified output: `[0, 0, 1, 1, 2, 2]` **Complexity:** O(n) time, O(1) space

I7

Count the current run of 1s; reset on 0; keep the maximum.

```
static int maxConsecutiveOnes(int[] a) {
    int best = 0, run = 0;
    for (int x : a) {
        if (x == 1) {
            run++;
            best = Math.max(best, run);
        } else {
            run = 0;
        }
    }
    return best;
}
```

Verified output: `3` **Complexity:** O(n), O(1)

I8

Same prefix-sum idea as Problem 18, but count how many earlier prefixes equal `sum - k`.

```
static int countSubarraysWithSum(int[] a, int k) {
    Map<Long, Integer> seen = new HashMap<>();             // prefix sum -> how many times
    seen.put(0L, 1);
    long sum = 0;
    int count = 0;
    for (int x : a) {
        sum += x;
        count += seen.getOrDefault(sum - k, 0);            // earlier prefixes that leave k
        seen.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Verified output: `2 | 4` **Complexity:** O(n) time, O(n) space

I9

Start with the widest container. The area is limited by the shorter line, so moving the taller line can never help; move the shorter one.

```
static int maxWater(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++;                              // the shorter wall limits the area
        else r--;
    }
    return best;
}
```

Verified output: `49` **Complexity:** O(n) time, O(1) space

I10

Add `m[i][i]` and `m[i][n-1-i]`. For odd n the centre cell is on both diagonals, so subtract it once.

```
static int diagonalSum(int[][] m) {
    int n = m.length, total = 0;
    for (int i = 0; i < n; i++) {
        total += m[i][i];                                  // primary diagonal
        total += m[i][n - 1 - i];                          // secondary diagonal
    }
    if (n % 2 == 1) total -= m[n / 2][n / 2];              // the centre was counted twice
    return total;
}
```

Verified output: `25 | 10` **Complexity:** O(n) time, O(1) space

A1

The water above a bar is `min(leftMax, rightMax) - height`. With two pointers we always process the side with the smaller wall, because that side's limit is already known.

```
static int trap(int[] h) {
    int l = 0, r = h.length - 1, leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (h[l] < h[r]) {
            leftMax = Math.max(leftMax, h[l]);
            water += leftMax - h[l];                       // left side is the limit
            l++;
        } else {
            rightMax = Math.max(rightMax, h[r]);
            water += rightMax - h[r];                      // right side is the limit
            r--;
        }
    }
    return water;
}
```

Verified output: `6 | 9` **Complexity:** O(n) time, O(1) space

A2

Put all numbers in a set. Only start counting at a number whose predecessor `x - 1` is missing, so each sequence is walked once.

```
static int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {
        if (!set.contains(x - 1)) {                    // x starts a sequence
            int len = 1;
            while (set.contains(x + len)) len++;
            best = Math.max(best, len);
        }
    }
    return best;
}
```

Verified output: `4 | 9` **Complexity:** O(n) time, O(n) space

A3

The answer is in `1..n+1`. Place every value `v` (with `1 <= v <= n`) at index `v - 1` by swapping. Then the first index `i` with `a[i] != i + 1` gives the answer. Each swap puts one value in its final place, so there are at most n swaps in total.

```
static int firstMissingPositive(int[] a) {
    int n = a.length;
    for (int i = 0; i < n; i++) {
        while (a[i] > 0 && a[i] <= n && a[a[i] - 1] != a[i]) {
            int t = a[a[i] - 1]; a[a[i] - 1] = a[i]; a[i] = t;   // put value v at index v-1
        }
    }
    for (int i = 0; i < n; i++) {
        if (a[i] != i + 1) return i + 1;
    }
    return n + 1;
}
```

Verified output: `2 | 3 | 1` **Complexity:** O(n) time, O(1) extra space (modifies the array)

A4

Keep a deque of indexes whose values are decreasing. Remove indexes that left the window from the front, and smaller values from the back. The front is always the maximum. Each index enters and leaves the deque once.

```
static int[] slidingMax(int[] a, int k) {
    int n = a.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();               // indexes, values decreasing
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();       // out of window
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();       // useless smaller
        dq.addLast(i);
        if (i >= k - 1) res[i - k + 1] = a[dq.peekFirst()];
    }
    return res;
}
```

Verified output: `[3, 3, 5, 5, 6, 7]` **Complexity:** O(n) time, O(k) space

A5

Sort. Fix the first number, then use two pointers on the rest. Skip repeated values to avoid duplicate triplets.

```
static List<List<Integer>> threeSum(int[] a) {
    int[] s = a.clone();
    Arrays.sort(s);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < s.length - 2; i++) {
        if (i > 0 && s[i] == s[i - 1]) continue;          // skip repeated first value
        int lo = i + 1, hi = s.length - 1;
        while (lo < hi) {
            int sum = s[i] + s[lo] + s[hi];
            if (sum < 0) lo++;
            else if (sum > 0) hi--;
            else {
                res.add(Arrays.asList(s[i], s[lo], s[hi]));
                lo++; hi--;
                while (lo < hi && s[lo] == s[lo - 1]) lo++;   // skip repeated values
                while (lo < hi && s[hi] == s[hi + 1]) hi--;
            }
        }
    }
    return res;
}
```

Verified output: `[[-1, -1, 2], [-1, 0, 1]] | [[0, 0, 0]] | []` **Complexity:** O(n^2) time, O(1) extra space (besides the copy and the output)